

Design and evaluate a Filtered Multi-Agent System with RAG for enterprise recommendation tasks

Master's Thesis submitted

to

Prof. Dr. Stephan Lessmann
Humboldt-Universität zu Berlin
School of Business and Economics
Institute for Business Administration
Chair of Information Systems

by

Davide Quaglio
(597201)

in partial fulfillment of the requirements
for the degree of
Master of Science

Berlin, September 2026

Declaration of Academic Integrity

I, Davide Quaglio, hereby declare that I have not previously submitted the present work for other examinations. I wrote this work independently. All sources, including sources from the Internet, that I have reproduced in either an unaltered or modified form (particularly sources for texts, graphs, tables and images), have been acknowledged by me as such. I understand that violations of these principles will result in proceedings regarding deception or attempted deception.

Signature



Place, date

Berlin, 25/09/2026

Abstract

Large Language Model (LLM) agents provide a promising method for carrying out complex analytical tasks, but the use of multiple agents leads to extra communication and computational demands. Although retrieval-augmented generation (RAG) and dedicated agent workflows have been extensively studied, it is still not clear whether the advantages of coordinated multi-agent work outweigh the extra cost and if this kind of overhead can be reduced by means of selective execution.

The thesis looks at this trade-off in an enterprise analytical context by comparing four different architectures: a single agent baseline, a retrieval-augmented single agent, a static multi-agent system together with RAG, and a Dynamic Filtered Multi-Agent system incorporating RAG. The architectures are assessed using 25 analytical questions, with three repetitions carried out under identical conditions regarding the model, the data, the semantic knowledge source, and the SQL environment. The dynamic architecture in question adjusts the level of agent involvement, the retrieved context, and the inter-agent communication in accordance with the demands of each individual question.

The experiments indicate that although static multi-agent coordination leads to better answer quality it also results in a considerable increase in computational cost. Dynamic filtering offers a more advantageous compromise since it improves both F1 and groundedness while at the same time reducing token usage and latency when compared to the static multi-agent approach. Yet its high rate of false abstention shows that too strict a filtering method can cause answer coverage to decrease. These findings demonstrate the potential of selective coordination for enhancing the efficiency of multi-agent analytical systems, while at the same time emphasizing the need for filtering and abstention mechanisms to be properly calibrated.

Contents

Declaration of Academic Integrity	1
1 Introduction	2
2 Related work	4
2.1 Large Language Models (LLMs)	4
2.2 Transformer Models	6
2.3 Transformer variants	8
2.4 Limitations of standalone LLMs	9
2.5 Supervised Fine-Tuning (SFT)	11
2.6 Retrieval-Augmented Generation (RAG)	12
2.7 LLM-Based Agent Systems	16
2.8 From single-agent to multi-agent LLM workflows	23
2.9 Multi-Agent Systems	24
2.10 Toward Efficient and Filtered Multi-Agent RAG	29
2.11 Positioning of this Thesis	33
3 Research Methodology	34
3.1 Research Design	34
3.2 Data and Experimental Setting	35
3.3 Agent Architectures	36
3.4 Experimental Procedure	44
3.5 Evaluation Metrics	44
4 Results	47
4.1 Overall Performance Across Architectures	47
5 Conclusion	51
5.1 Limitations and future work	52

1. Introduction

Large language models (LLMs) are increasingly being considered for enterprise recommendation and decision-support tasks because they can process large volumes of text, synthesize information, and generate coherent task-oriented output. This thesis, *Design and Evaluation of a Filtered Multi-Agent System with RAG for Enterprise Recommendation Tasks*, investigates how retrieval-augmented and agentic workflow designs can improve the reliability and cost-efficiency of LLM-based recommendation systems in enterprise settings.

Despite their capabilities, the practical deployment of LLMs in enterprise environments remains challenging. Their use is constrained by several well-known limitations, including hallucinations, brittleness under distribution shifts, weak causal reasoning, and difficulties with long-horizon planning.

These weaknesses become particularly critical when enterprise tasks require the integration of domain-specific and heterogeneous data sources. In such settings, standard prompting alone is often insufficient to ensure grounded and reliable recommendations (Y. Gao, Xiong, X. Gao, et al. 2024). One major challenge is hallucination, that is, the generation of plausible but factually unsupported content when relevant infor-

mation is missing, ambiguous, or outdated. This is especially problematic in enterprise recommendation tasks, where the output must be traceable to verifiable evidence rather than only to patterns learned during pretraining.

Enterprise workflows further increase this difficulty because relevant knowledge is rarely contained in a single source. Instead, it is typically fragmented across structured business records, internal documents, domain-specific terminology, and evolving organizational knowledge that a base model does not reliably know. In addition, LLMs can be brittle under distribution shifts, as they often encounter customer profiles, behavioral patterns, and product configurations that differ from those represented in their training data. Their reasoning is also limited when tasks require distinguishing causal relations from superficial correlations or maintaining consistency in multiple sequential steps. As a result, enterprise recommendation workflows present a particularly demanding setting for LLM-based systems.

Current research suggests that techniques such as Retrieval Augmented Generation (RAG) (Arslan, Munawar, and Cruz 2024; L. Chen et al. 2025) and specialized multi-agent systems (Arrais et al. 2026; Y. Wang, J. Wang, and Chu 2025) may help address these limitations. RAG improves LLM performance by conditioning the generation on evidence re-

trieved from external corpora, thus increasing factual support, domain adaptation, and traceability (P. Lewis, Perez, Piktus, Petroni, Karpukhin, Goyal, Kuttler, et al. 2020; Y. Gao, Xiong, X. Gao, et al. 2024). Instead of relying exclusively on parametric memory, the model can ground its output in enterprise-specific knowledge sources, making recommendations more robust and transparent.

In parallel, the shift from monolithic, single-pass prompting toward Multi-Agent Systems (MAS) represents an important development in the design of LLM-based applications. Often discussed under the broader notion of agentic AI, these systems decompose complex tasks into specialized subtasks, such as retrieval, analysis, and synthesis. This decomposition can improve the quality of reasoning by distributing work across distinct roles and structuring intermediate outputs more explicitly. For enterprise recommendation tasks, such architectures are particularly promising because they can combine heterogeneous evidence sources and support, more deliberate multi-step reasoning than a single-agent design (Prasad et al. 2024) and (C. Zhang et al. 2024).

Nevertheless, such benefits come with a price. MAS architectures typically introduce additional communication overhead, increase token consumption, and lead to longer response times, as intermediate results must be

exchanged, interpreted, and reformulated between agents (Xi et al. 2025). In enterprise deployments, where scalability, latency, and operational cost are critical, such inefficiencies can outweigh the benefits of improved response quality. This trade-off motivates the central problem of the thesis: whether a filtered multi-agent architecture can preserve the reasoning advantages of MAS while reducing the communication inefficiencies that limit its practical use.

In summary, this thesis makes three primary contributions. Firstly, it suggests a dynamic filtering multi-agent architecture using retrieval augmented generation (RAG) that aims to preserve the reasoning benefits of multi-agent collaboration while reducing unnecessary communication and computational overhead. Second, it provides a comparative study of four increasingly sophisticated architectures: a Single-Agent System (**SAS**), a recovering assisted Single-Agent System (**SAS_RAG**), a Retrieval-assisted Multi-Agent system (**MAS_RAG**), and a Dynamically Filtered Multi-Agent System with RAG (**Dynamic MAS_RAG**). To ensure a controlled comparison, all four architectures are evaluated under identical conditions with respect to the underlying model, data, semantic knowledge, and evaluation procedure. Lastly, the research provides an empirical understanding of the trade off be-

tween quality and efficiency in multi-agent collaboration, whereby token consumption can be minimized without compromising quality or grounding. However, the dynamic architecture also shows a higher tendency to abstain from answering questions, highlighting the need to balance efficiency and reliability with sufficient answer coverage.

2. Related work

The rapid development of large language models has significantly influenced the design of contemporary AI systems for language understanding, generation, and decision support. However, their application in organizational contexts raises additional challenges related to reliability, efficiency, and effective use of external knowledge. This section reviews the main research areas relevant to the thesis, with a focus on how recent work addresses grounding, coordination, and adaptive reasoning in LLM-based systems. The discussion moves from foundational concepts to approaches that are directly relevant to retrieval augmented generation and multi-agent architectures.

2.1 Large Language Models (LLMs)

Evolution and fundamentals of large language models

Large language models (LLMs) have evolved from research prototypes to core components of many contemporary AI applications. Different models have been published and implemented as support in different fields, such as finance, education, agriculture, healthcare, and many more (X. Chen et al. 2025). Figure 1 provides an overview of the development of large language models, tracing their evolution from the earliest approaches to contemporary LLM architectures. Earlier language modeling approaches were dominated by statistical methods, particularly n-gram models, which estimate the probability of a word based on a fixed number of preceding words. The simplicity of n-gram models was to predict the probability of the next word from the previous n-1 words. However, the fixed window of this model struggled with data sparsity and long-range relations (Cavnar and Trenkle 1994).

With the evolution of Word Vectors and Recurrent Sequence Models (RNNs) the transition from statistical models to neural representation marked a shift. RNNs and ELMo (Peters et al. 2018) could better transfer knowledge to new tasks and outper-

formed n-grams in terms of robust semantics. Nevertheless, the introduction of sequential memory and dynamic embeddings still created a computational bottleneck that limited the ability to handle long-range dependencies required for modern and complex LLM workflows.

The current era of language modeling shifted from sequential processing to the Transformer architecture, to enable parallelization and the capture of long-range dependencies which the previous models were not able to fully capture (Vaswani et al. 2017). The Transformers architecture was followed by multiple researches such as (Devlin et al. 2019) with the BERT model and (Radford et al. 2018) with

GPT models and culminated with the release of chatbots like ChatGPT from OpenAI.

Since that release, researcher followed the "scaling law" described by (Kaplan et al. 2020) which demonstrated that by increasing the model parameters and the amount of training data, the model performance often increases. This approach was followed in order to shift from an early primary text generation model to a development of models which can tackle a wide range of tasks.

Defining large language models

Large language models are large-scale neural language models trained on extensive text corpora, including books,

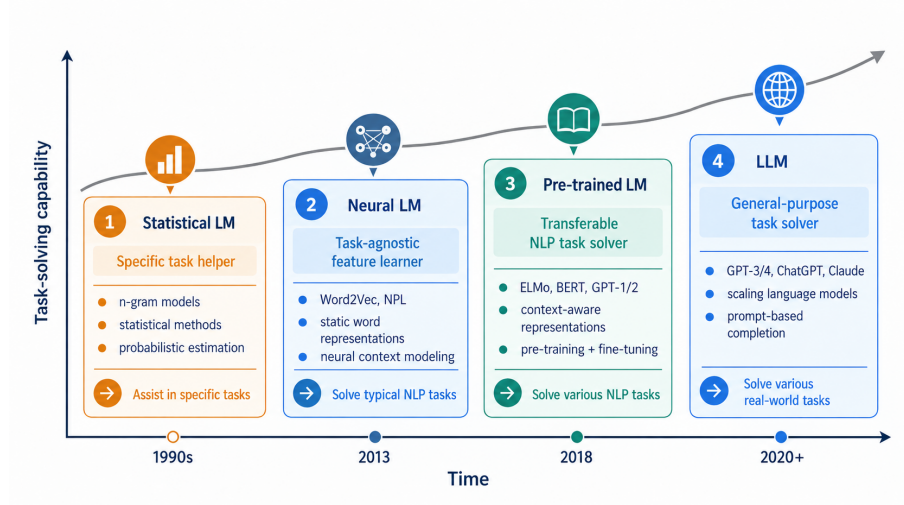


Figure 1: The figure illustrates the evolution from statistical language models to modern LLMs, highlighting the increasing task-solving capabilities of each generation.

code, scientific documents, and web data. Although the exact parameter threshold for an LLM is not strictly defined, the term typically describes models that starting at the billion-parameter scale and extend to trillions (Kaplan et al. 2020). As previously explained, LLMs are neural networks built on Transformers architectures, which utilizes the same fundamental principles of traditional machine learning models to train and optimize model parameters but with more general goals: LLMs aim to provide general-purpose language capabilities that can be adapted to a wide range of downstream tasks. To achieve this objective, the training process is divided into two main stages: pre-training and post-training (W. X. Zhao et al. 2026). The pre-training phase is the most critical and computational intensive stage, where the model learns which general patterns of language from very large amounts of text. Starting from randomly initialized parameters, this process yields a general-purpose base model that captures syntax, semantics, and broad factual associations. After pre-training, the post-training phase adapts this base model to practical use through techniques such as instruction tuning and alignment, which improve its ability to follow human instructions and generate outputs that better reflect user expectations. In contrast to pre-training, post-training typically requires substantially less

data and computational resources.

2.2 Transformer Models

This section introduces the Transformer architecture as the dominant neural architecture underlying contemporary LLMs. The architecture relies primarily on input embeddings with positional information, multi-head attention, feed-forward networks, residual connections, and layer normalization.

Input Encoding Layer

The input encoding layer transforms the sequence of discrete tokens into numerical representations that can be processed by the Transformer. Each token is first assigned to a dense embedded token v_t . Since the Transformer processes tokens largely in parallel and does not inherently encode word order, positional information p_t is incorporated to indicate the position of each token within the sequence. The final input representation x_t is therefore obtained by combining token and positional embeddings, as shown in Eq. (1).

$$x_t = v_t + p_t \quad (1)$$

These representations are subsequently passed to the attention layers, where information from different positions in the sequence is combined to produce context-dependent representations.

Multi-head Attention layer

The multi-head attention layer is a core mechanism of the Transformer model and consists of multiple attention modules, commonly referred to as attention heads. In each attention head, the input embedding sequence X is projected into three representations: the query matrix Q , the key matrix K , and the value matrix V . These representations are obtained by multiplying the input sequence with the learned weight matrices W_Q , W_K , and W_V respectively, as shown in Eq.(2).

$$\begin{aligned} Q &= XW^Q, \\ K &= XW^K, \\ V &= XW^V. \end{aligned} \quad (2)$$

The query and key matrices are

then used to compute the attention scores through dot-product similarity. These scores indicate the relevance of each token to each other in the sequence.

To obtain normalized attention weights, dot-product scores are scaled by the square root of the key dimension $\sqrt{D}$ and passed through a softmax function. The scaling operation prevents the dot products from becoming excessively large, which could otherwise lead to unstable gradients and overly sharp softmax distributions. The resulting attention weights are then applied to the value matrix V , producing a contextualized representation of the input tokens. Formally, this operation is expressed in Eq.(3):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{D}}\right) V \quad (3)$$

$$\text{MHA} = \text{Concat}(\text{head}_1, \dots, \text{head}_N) W^O \quad (4)$$

In multi-head attention, this process is performed multiple times in parallel by several attention heads. Each head uses its own parameter matrices and can therefore learn different types of relationship within the sequence. For instance, one head may capture syntactic dependencies, while another may focus on semantic associations or long-distance token interactions. The outputs of the indi-

vidual heads, expressed in Eq.(4) are concatenated and subsequently transformed through an output projection matrix W^O . This allows the model to integrate multiple representational perspectives into a single output representation.

This approach provides a significant advantage over recurrent architectures such as RNNs, LSTMs, and GRUs, as these models process in-

put sequences step by step and can have difficulty maintaining information across long distances. In contrast, attention mechanisms allow computations to be performed in parallel, making Transformer models well suited for modern hardware accelerators such as GPUs and TPUs (Vaswani et al. 2017) and (Al-Hammuri et al. 2023). This ability to parallelize computations is a major factor that enables Transformers to scale effectively to large datasets and increasingly complex model architectures (X. Chen et al. 2025) and (W. X. Zhao et al. 2026).

Feed-Forward Network Layer

After the multi-head attention mechanism has captured contextual dependencies between tokens, the FFN applies additional nonlinear transformations to each token representation independently.

$$\text{FFN}(x) = \sigma(xW_1 + b_1)W_2 + b_2 \quad (5)$$

In Eq.(5) where W_1 and W_2 are learned weight matrices, b_1 and b_2 are bias terms, and σ denotes a nonlinear activation function such as ReLU or GELU. Typically, FFN consists of two linear transformations with an activation function, such as ReLU, between them. This nonlinear transformation increases the representational capacity of the model and enables it to capture more complex patterns in the data.

Therefore, while the attention mechanism models relationships between tokens, the FFN further refines contextualized representations before they are passed to the next Transformer layer (Vaswani et al. 2017).

Residual connections and layer normalization

Residual connections and layer normalization are used throughout the Transformer to support stable information flow across layers. Residual connections preserve the original representation by adding it to the output of a sublayer, while layer normalization stabilizes the resulting activations. Together, these mechanisms improve training stability and enable the construction of deep Transformer architectures (Ba, Kiros, and Hinton 2016).

2.3 Transformer variants

Transformer-based models are commonly grouped into three architectural variants: encoder-only, decoder-only, and encoder-decoder models. These variants differ in their attention mechanisms, training objectives, and suitability for downstream tasks. Each variant introduces different trade-offs with respect to language understanding, sequence generation, latency, and task flexibility.

Encoder

The encoder architectures are composed of stacking multiple identical layers, containing the multi-head attention (MHA) and the feed-forward network (FNN) modules. These layers model contextual semantic information and generate the encoded representation for the given input. This encoder is bidirectional, allowing all input embeddings to be fully visible to each other, enabling each position to aggregate semantic information from the entire input sequence (Devlin et al. 2019) and (Z. Liu et al. 2023). These models are particularly effective when the expected output is a label, therefore they can be efficient for classification, retrieval, ranking, and reranking tasks under strict latency constraints (X. Chen et al. 2025) and (W. X. Zhao et al. 2026).

Decoder

The decoder architectures are well suited for open-ended generation, instruction following, dialog, tool use, and in-context learning. They incorporate a masked multi-head attention mechanism which prevents future information from being leaked when the output sequence is generated.

Decoder-only architectures, such as GPT-style models and BLOOM, are optimized for autoregressive language generation. They predict the

next token based on the preceding context and generate output sequentially from left to right. This makes them well suited for open-ended generation, instruction follow-up, dialog, tool use, and in-context learning. However, their inference cost depends on the number of tokens generated, the speed of decoding, the efficiency of the cache, and the potential overhead of calls to external tools.

2.4 Limitations of standalone LLMs

Despite their strong capabilities in language understanding, generation, and generalization, standalone Large Language Models (LLMs) exhibit several limitations when applied to knowledge-intensive and domain-specific tasks. A central limitation is their tendency to generate fluent but factually incorrect or unsupported outputs, a phenomenon commonly referred to as hallucination. Hallucination has been widely discussed in natural language generation and is particularly problematic in real-world applications, where generated content must be reliable, verifiable, and grounded in available evidence (Ji et al. 2023; Yue Zhang et al. 2023).

Another limitation concerns the way knowledge is stored and accessed in pretrained language models. Much of the knowledge of an LLM is encoded implicitly in its parameters,

making it difficult to update, inspect, or trace back to specific sources. As a result, standalone LLMs can rely on outdated information and provide limited provenance for their generated outputs (P. Lewis, Perez, Piktus, Petroni, Karpukhin, Goyal, Küttler, et al. 2020). This becomes especially problematic in organizational contexts, where relevant information is often dynamic, domain-specific, and distributed between internal documents, databases, APIs, and operational systems.

Furthermore, LLMs are trained on large-scale corpora that can contain bias, incomplete information, or unfiltered textual patterns. Therefore, their outputs may reflect limitations of the underlying training data rather than verified domain knowledge (Bender et al. 2021). In enterprise environments, these limitations reduce the suitability of standalone LLMs for tasks that require factual accuracy, transparency, and up-to-date knowledge.

Retrieval-Augmented Generation (RAG) addresses these limitations by combining the parametric knowledge of pretrained language models with external, retrievable knowledge sources. Instead of relying solely on knowledge encoded during pre-training, RAG systems retrieve relevant documents or passages at inference time and use them to condition the generated output. This enables more grounded,

updateable and traceable generation, making RAG particularly relevant for domain-specific and knowledge-intensive applications (P. Lewis, Perez, Piktus, Petroni, Karpukhin, Goyal, Küttler, et al. 2020; Y. Gao, Xiong, X. Gao, et al. 2024).

Hallucination

LLM have been trained on a huge amount of text data, which have possibly included outdated and biased information. Moreover, the strong abilities of LLMS may generate false information which seems highly correct and accurate, which is difficult to detect also for human, even in fields such as medicine and law. This research (Yue Zhang et al. 2023), categorizes hallucination of LLMs into 3 groups: Input-conflicting hallucination arises when there is a contradiction between the generated answer and the input of the task, by misunderstanding the human input. Current LLM still suffer from this hallucination, especially in the long text scenario. Context-conflicting hallucination arises when the model demonstrates a lack of consistency throughout the conversation by exhibiting self-contradictions over multi responses. Fact-conflicting hallucination occurs when the generated content is inconsistent with established world knowledge and is also the focus of recent research, since it tends to have more

side effects.

Additional Challenges of LLMs

However, hallucination is seen as the central problem for LLMs, and there exist other limitations that are less painful but still present and almost unperceptable. Ambiguity refers to a response which has multiple interpretations and it is not necessarily incorrect, but does not fulfill the user’s information needs (Tamkin et al. 2022). Incompleteness occurs when the generated response has been truncated resulting in an incomplete explanation (Bai et al. 2025). LLMs are in general biased because they have been trained on sources that may mirror social biases, gender bias or discriminatory beliefs (Navigli, Conia, and Ross 2023). The last limitation discussed by (Y. Gao, Xiong, X. Gao, et al. 2024) is under-informativeness, where LLMs have the propensity to avoid answering certain questions however they have the capability. This state of under-informativeness may have occurred because of an over-optimization during Reinforcement Learning (RLHF). These limitations have motivated different approaches for adapting pretrained language models to specific tasks and expected behaviors. One of the most established approaches is Supervised Fine-Tuning (SFT).

2.5 Supervised Fine-Tuning (SFT)

A major issue encountered during the previous stages of the LLMs development was the mismatch between the training objective and the user objective. Before RAG techniques have been introduced as a partial solution, the researchers Zhang et al. (S. Zhang et al. 2025) introduced Supervised fine-tuning (SFT). This technique enables a more controllable behavior of the model compared to standard LLMs, by introducing human instructions, which constrain the output of the model and align it with the desired domain knowledge. However, this approach is computationally efficient and helps a rapid adaptation of the model to a specific domain avoiding retraining, it poses specific challenges. Researchers concern that crafting instructions that cover the desired behavior is non-trivial, since the instruction datasets are limited in diversity and creativity (understand this part).

A major challenge in the development of LLMs is the mismatch between the general objective used during pretraining and the behavior expected by users for specific tasks. Supervised Fine-Tuning (SFT) addresses this problem by further training a pretrained model on examples consisting of instructions and corresponding desired responses. Zhang et al. (S.

Zhang et al. 2025) discuss this type of instruction-based adaptation as a mechanism to improve the controllability and task-specific behavior of language models.

Through SFT, a general-purpose model can be adapted to particular tasks or domains without being trained again from scratch. However, its effectiveness is strongly dependent on the quality and diversity of the instruction data. Constructing datasets that adequately represent the range of desired behaviors is difficult, and limited training examples may result in poor generalization to instructions that differ from those observed during fine-tuning.

Furthermore, because knowledge acquired through SFT is encoded within the model parameters, updating or extending this knowledge generally requires additional fine-tuning. Retrieval-Augmented Generation provides a different approach by supplying relevant external information at inference time rather than relying exclusively on knowledge stored within the model.

2.6 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is a promising solution technology which enables the gathering of external data, ensuring more accurate and updated responses given

a certain input. This approach is more cost effective than training and fine tuning the model again with fresh information, recognizing RAG as a flexible and scalable solution (Y. Huang and J. X. Huang 2026). This section is meant to clarify this technology, by explaining in details the steps of the RAG framework.

(Y. Gao, Xiong, X. Gao, et al. 2024) and (Y. Gao, Xiong, M. Wang, et al. 2024) propose a review of the state of the art RAG methods, including naive RAG, advanced RAG, and modular RAG. Figure 2 provides an overview of these three RAG paradigms and highlights how retrieval and generation become progressively more structured and modular. The Naive RAG is the earliest methodology and based on three main components for a given user query $Q = \{q_i\}$. The indexing step is performed before inference and where raw external knowledge is cleaned and chunked. Given a document collection $D = \{d_1, d_2, \dots, d_n\}$, each document is first divided into smaller text segments, commonly referred to as document chunks. Each chunk d_i is then transformed into a dense vector representation using an embedding model $f_e(\cdot)$. The resulting embeddings are converted into vector representations for storage enabling rapid, precise, and efficient similarity search during the retrieval phase. Formally, the indexed representation can be expressed

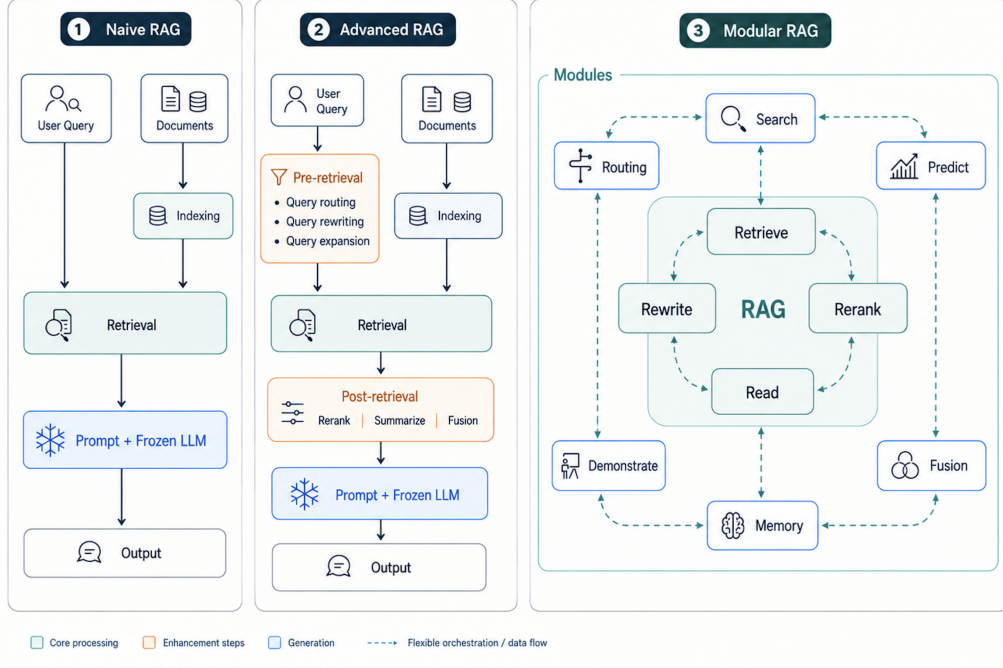


Figure 2: Retrieval-Augmented Generation (RAG) overview. The figure compares three RAG paradigms, illustrating the transition from linear retrieval-generation pipelines to more flexible and modular architectures.

as follows.

$$I = \{e_1, e_2, \dots, e_n\}, \quad e_i = f_e(d_i)$$

where e_i denotes the embedding vector of the document chunk d_i , $f_e(\cdot)$ is the embedding model and d represents the dimensionality of the embedding space.

The retrieval stage transforms the user query q into a dense vector representation e_q using the same embedding model. Afterward, vector similarity measures, such as the dot product shown in Eq. (6) or cosine similarity

shown in Eq. (7), are used to calculate the similarity score between the query representation e_q and the document representations e_{d_i} .

$$\text{Sim}(e_q, e_{d_i}) = e_q \cdot e_{d_i} \quad (6)$$

or

$$\text{Sim}(e_q, e_{d_i}) = \frac{e_q \cdot e_{d_i}}{\|e_q\| \|e_{d_i}\|} \quad (7)$$

The result of this phase, as shown in Eq. (8), are the retrieved chunks $D_q = \{d_1, d_2, \dots, d_k\}$, which have

been selected as the top- k chunks that are closest to the query in the embedding space.

$$D_q = \text{topk}_{d_i \in D} \text{Sim}(e_q, e_{d_i}) \quad (8)$$

The last phase is the generation stage, where the queries q and the relevant document D_q chunks are provided to the LLM to generate the final response, as shown in Eq. (9).

$$y = \text{LLM}([D_q, q]) \quad (9)$$

Despite its utility, Naive RAG is limited, as it often struggles with precision and recall, persistent hallucinations during generation, and redundant or disjointed output during augmentation. These hurdles necessitate the transition to Advanced RAG architectures, which have been implemented with four main phases: *pre-retrieval*, *retrieval*, *post-retrieval*, and *generation* (Y. Huang and J. X. Huang 2026).

During the *pre-retrieval* stage, the focus is on two goals: Optimizing and enhancing the quality of indexing and of the original query, which can be achieved using mainly techniques like query manipulation and data modification. *Query manipulation* seeks a better match between the indexed data and the user queries, which includes query reformulation, query expansion, and query normalization. In query reformulation, the

original user query is rewritten to improve its clarity or retrieval effectiveness. Query expansion enriches the query with synonyms, related terms, or alternative expressions, while query normalization resolves differences in spelling, terminology, or formatting. The second task of pre-retrieval is *data modification*, including pre-processing techniques such as removing irrelevant or redundant information to improve data quality or enriching the data with metadata (X. Ma et al. 2023), (Peng et al. 2023), (L. Gao et al. 2023). The *retrieval* phase follows the same general procedure as in the Naive RAG. However, its effectiveness is improved by the additional *preretrieval* processing steps. Since the retrieval mechanism itself remains unchanged, its detailed explanation is omitted here and can be referred to in the previous section.

The *post-retrieval* phase consists of re-ranking and filtering the document selection before the generation phase. The re-ranking step incorporates additional metrics and external sources to accurately highlight and reorganize the previously retrieved documents, by diminishing the importance of less relevant ones. The filtering step eliminates documents that did not exceed the minimum relevance threshold (Xiangji Huang and Q. Hu 2009), (Y. Huang and J. X. Huang 2023), (Khat-tab and Zaharia 2020).

The *generation* phase encompasses two subtasks: enhancing and cus-

tomization. The enhancement step improves the quality of the output by rephrasing and organizing the information. Customization wants to tailor the final answer to the user, by shaping the response to be contextually relevant user-centric.

The latest version, *modular RAG*, is an extension of the principles of the RAG family, which breaks the rigid sequence of the two previous versions, by treating every component as independent and interchangeable that can be reordered depending on the task. This modular structure is formalized in Eq. (10), where the available modules and their configurations can be arranged into different RAG flows. Introduces additional specialized components such as Search, Memory, Fusion, and Routing modules to offer enhanced adaptability and versatility. This reduces irrelevant noise and ensures that the information retrieved is more accurate and flexible for complex problems (Y. Huang and J. X. Huang 2026). For instance, the Search module uses the LLM to automatically write the right query and enable the search of external information consulting the web, external APIs, and databases. The Fusion module expands the user query by creating a multi-query strategy, runs them in parallel, and combines and re-ranks the results intelligently. The memory module creates a memory pool that remembers previous iteration to guide

the next retrieval search. The Routing module redirects the incoming queries to the most suitable pipeline, enabling the system to adapt its behavior instead of relying on a single and fixed retrieval flow (Singh et al. 2025) and (P. Liu, X. Liu, J. Ma, et al. 2025).

$$\begin{aligned}\mathcal{G} &= \{q, D, \mathcal{M}, \{\mathcal{M}_s\}, \{Op\}\}, \\ \mathcal{F} &= (M_{\phi_1}, \dots, M_{\phi_n})\end{aligned}\quad (10)$$

Notation.

q user query.

D document repository.

$\mathcal{M}$ set of RAG modules.

$\mathcal{M}_s$ sub-modules.

Op operators implementing specific functions.

M_{ϕ_i} module i with configuration ϕ_i .

In addition, the standard retrieval setting, Gao et al. (Y. Gao, Xiong, X. Gao, et al. 2024) introduced that RAG can also be extended through three broader retrieval strategies. First, iterative retrieval alternates the retrieval and generation steps, so that the model can progressively obtain more relevant and targeted context from the knowledge base. Second, recursive retrieval refines the initial query over several stages by decomposing a complex problem into smaller sub-problems, which are then solved step

by step through repeated retrieval and generation. Third, adaptive retrieval enables the system to decide autonomously whether external retrieval is needed and when the retrieval-generation process should stop, often relying on control signals produced by the LLM itself.

However, Arslan et al. (Arslan, Ghanem, et al. 2024) improves the fact that the drawback of deploying RAG with large datasets with varying structures (e.g., structured, semi-structured, and non structured) may lead to processing delays. For example, in the biomedical sector, there is the risk that RAG exposes private information from specific clients to stakeholders that should not be able to access, violating client confidentiality.

Vyas et al. (Vyas et al. 2024) customized a model with RAG as a solution to help technicians search for specific technical industrial knowledge, such as internal regulation and product maintenance. This knowledge system has been developed to reduce the response time of employees when responding to discussions with customers. The authors implemented a multi-stage pipeline combining BM25 keyword matching and vector embeddings for retrieval, followed by a BAAI reranker, to allow technicians to interactively query 234 MB of heterogeneous industrial data such as PDFs, CSVs, and technical manuals. Their

results showed that the hybrid approach achieved high precision, with recall reaching up to 91% for regulatory documents, while providing a more sustainable and traceable alternative to the static and potentially hallucination-prone knowledge of standard LLMs in industrial settings.

2.7 LLM-Based Agent Systems

Traditional agent systems have already been researched and applied with the goal of solving problems or performing tasks automatically. However, their reliance on heuristic rules and static knowledge bases limited their adaptability and scalability in dynamic scenarios (Dietrich et al. 2003). In this context, the ability of LLMs to address complex tasks has enabled their integration as a central component in agent systems, allowing agents to exhibit more human-like decision-making capabilities.

This subsection focuses on the construction of an LLM-based Single-Agent System (SAS), where one agent uses internal modules such as profiling, memory, planning, and action to perform tasks autonomously (L. Wang et al. 2024). The main difference between a chatbot and an agent is that the latter is expected not only to respond to user input, but also to perform tasks through self-directed planning and actions. For this rea-

son, architectural design becomes a key element. The following part describes the main components of an SAS, namely the *profiling module*, the *memory module*, the *planning module*, and the *action module*. Their interaction within a representative single-agent architecture is illustrated in figure 3.

Before these modules can operate, the agent first needs to perceive input from its environment. In an SAS, perception refers to the information received by one individual agent, such as user instructions, environmental observations, tool output, or previous conversational context. This perceived information becomes the basis for memory retrieval, planning, and execution of action. Therefore, perception can be understood as the entry point of the agent loop, as the agent requires external or contextual information before deciding how to act.

Profiling module.

The first module, the *profiling module*, has the goal of defining the role of the agent, which influences the behavior of the LLM. There are different methods for profiling an agent. In *handcrafted profiling*, agent profiles are manually specified by humans. The *LLM-generation method*, instead, automatically creates profiles based on rules provided by humans. Although this method can be efficient, it may provide less control over the

generated profiles and may drift from what the user actually intended. Finally, the *data set alignment method* uses real data to transform each person’s data into an agent profile.

Memory module.

The *memory module* plays a crucial role in the architecture of an agent, as it allows the agent to store experiences, adapt over time, and behave more reasonably. The idea is to simulate how human memory works using short- and long-term memory. In agents, short-term memory corresponds to the context window that the model can access during inference. However, since this memory has limited duration and capacity, relevant information can be transferred to long-term memory if needed. Long-term memory, on the contrary, is a persistent repository of information retained by the agent over extended periods. It can contain factual knowledge, past experiences, and fundamental skills (L. Wang et al. 2024). The LLM retrieves this information through a retrieval component and can update long-term memory through a reflection process (Shinn et al. 2023). This module is therefore essential for enabling the following modules to work effectively.

Memory operations can be classified into three main categories: *memory retrieval*, *memory storage*, and *memory reflection* (L. Wang et al. 2024). Through *memory retrieval*,

valuable information related to a specific situation is extracted from the agent’s memory with the goal of improving decision-making accuracy. There are two main scenarios. When short-term memory is retrieved, the available context is directly accessed. Instead, before extracting content from long-term memory, a filtering mechanism is implemented to select the most relevant information and avoid inefficient behavior. To maintain the flexibility and adaptability of agents, memories are retrieved automatically using different approaches. Earlier approaches evaluated memory information based on defined metrics

such as recency, relevance, and importance (Park et al. 2023). Later approaches used embeddings and vectors, or reinforcement learning, to fine-tune the model for better task adaptation (Qian et al. 2024). In certain studies, users have also been allowed to manipulate and delete specific dialog history from agent memory, providing more control over the memory systems of LLM-based agents (C. Hu et al. 2023).

Memory storage is essential to allow the agent to act in a rational manner. Therefore, perceived information and learned experiences are archived in natural language text in memory

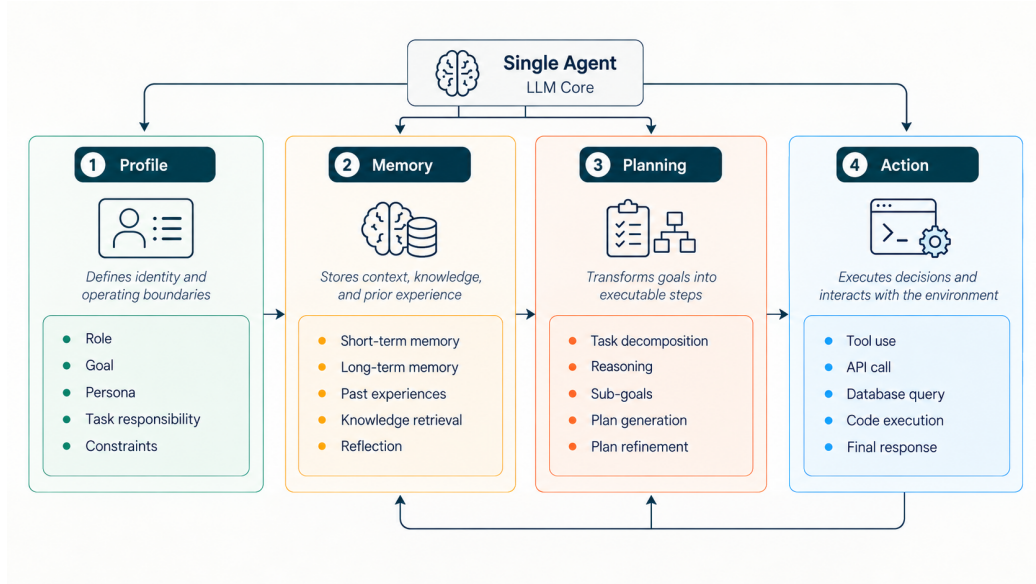


Figure 3: Single-Agent System (SAS) architecture. The figure summarizes the main components of a single-agent system, including profile, memory, planning, and action, coordinated by an LLM core.

following a specific approach. This approach ensures that the most relevant data are ready to be accessed while avoiding inefficient memory retrieval during interaction. In addition, adapting the storage format to the specific task has shown improvements in performance in various environments (Z. Huang et al. 2023). Furthermore, the integration of LLMs with databases has enabled efficient interfaces to manage and query stored data, while also improving the overall efficiency and usability of the system (Zhou et al. 2023).

Although memory storage improves the agent’s decision-making process, it also introduces two main challenges that need to be addressed. First, while the agent interacts with the environment, new information may need to be stored in memory. However, the incorporation process needs to be carefully considered in order to understand whether the information should be added, merged with existing information, or used to substitute erroneous data. If the accumulated context exceeds the predefined limit, the LLM may compress the text into a more compact representation. Second, since the memory storage of an agent is limited, choosing an appropriate strategy to exchange information is crucial. RET-LLM uses a first-in-first-out (FIFO) strategy, which overwrites the oldest data with the newest data in a limited-

size memory (Modarressi et al. 2024). In contrast, ChatDB deletes irrelevant information to increase available memory space (Zhou et al. 2023).

Memory reflection is another process that emulates the human practice of summarizing, refining, and reflecting upon existing knowledge. Through this process, the agent can revise previously stored information, update its knowledge, and improve the consistency of future decisions. Therefore, memory reflection supports the agent in maintaining a more coherent and useful memory over time.

Planning module.

The *planning module* is one of the most critical parts of agents, since it requires advanced understanding, reasoning, and decision-making processes. It refers to the process of deciding which action to take in order to achieve a goal by observing the environment and selecting the next action as part of a sequence. Previous methods, such as Planning Domain Definition Language (PDDL) (Haslum et al. 2019) or policy learning (Yao, Rao, et al. 2020), have been used for agents, but they also present limitations. Symbolic methods such as PDDL often require formal models that rely on human expertise, while reinforcement learning algorithms usually require a large number of interactions with the environment to learn an effective policy. This

makes them costly and impractical in scenarios where data collection is expensive. Therefore, the advanced reasoning capabilities of LLMs make them suitable as the cognitive core of agents. LLM-based agent planning can be categorized into five tasks, including *task decomposition*, *multi-plan selection*, *external module-aided planning*, *reflection and refinement*, and *memory-augmented planning* (Xu Huang et al. 2024).

Task decomposition breaks down the complex task of the agent into smaller and more manageable tasks, improving the agent’s ability to complete the overall task. This task includes two crucial steps, which are depicted in Figure 4: the decomposition step, which decomposes the initial main task into sub-goals, and the sub-plan step, which plans the goal of each sub-task. By decomposing the main task, the method can reduce the risk of task forgetting and hallucination by creating a stronger connection between sub-tasks (Touvron et al. 2023). However, task decomposition also introduces additional overhead, since the agent needs to perform more reasoning steps, which increases computational costs.

Despite the strong reasoning ability of LLMs, a single generated plan can be suboptimal or even infeasible, especially for complex tasks. For this reason, *multiplan selection* is used. This task includes two major steps.

The first one is multi-plan generation, where the LLM generates multiple possible planning paths. Several approaches can be adopted, such as Self-consistency, which assumes that solutions for complex problems are rarely unique. Instead of generating only one reasoning path, it generates multiple reasoning paths through sampling techniques such as temperature sampling and top-k sampling. Tree-of-Thought (ToT) generates multiple possible plans, either by sampling them during decoding or by explicitly asking the LLM to propose different options through examples (Yao, D. Yu, et al. 2023). Graph-of-Thought (GoT) extends this idea by allowing thoughts to be transformed and combined, while LLMs-MCTS and RAP use LLMs to guide the Monte Carlo Tree Search by suggesting possible actions (Besta et al. 2024; Z. Zhao, W. S. Lee, and Hsu 2023).

Once several candidate plans have been generated, different selection strategies can be used to identify the most suitable one. Self-consistency uses majority voting, ToT applies Tree Search methods such as BFS and DFS while the LLM evaluates the most promising option, LLM-MCTS and RAP use Monte Carlo Tree Search, and LLM A* relies on the classic A* algorithm to guide the planning process.

Multi-plan selection is an advanced technique that explores multi-

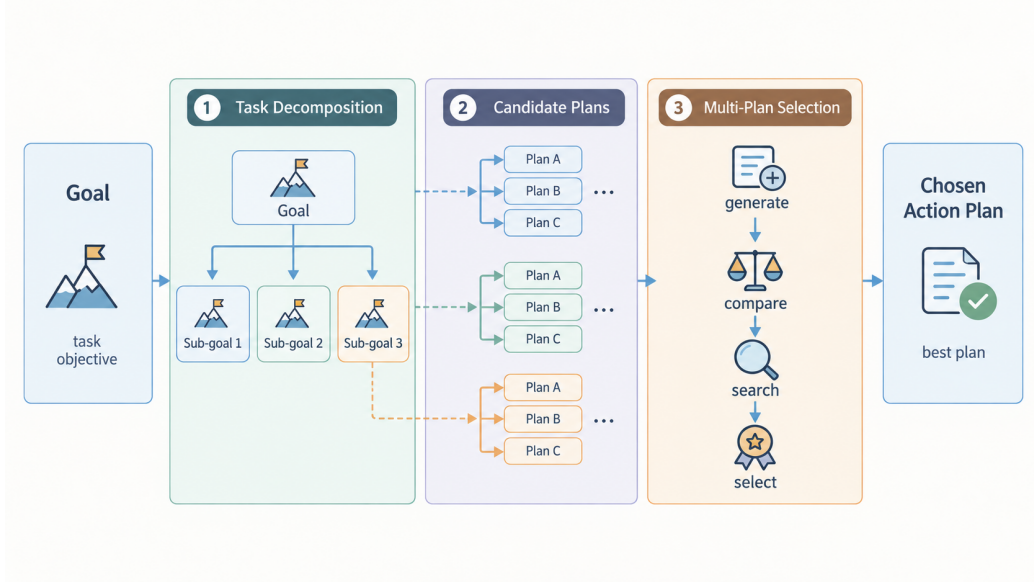


Figure 4: Task decomposition and multi-plan selection in LLM-based agent planning.

ple solutions in an expansive search space. However, this benefit comes with certain trade-offs. Increased computational demand is one factor that needs to be considered. Moreover, since LLMs naturally introduce randomness into the selection process, the evaluation of plans can affect the consistency and reliability of the final choice.

In addition to generating and selecting plans internally, LLMs can also be supported by external planners. This is particularly relevant when the environment contains strict rules or complex constraints. For this reason, some approaches combine LLMs with external planners, which can be either

symbolic planners or *neural planners*. Symbolic planners use formal models such as PDDL to find valid plans from an initial state to a goal state (McDermott et al. 1998; Hashum et al. 2019). In these approaches, the LLM usually translates the task into a formal planning language, while the external planner solves the actual planning problem. Examples include LLM+P, LLM-DP, LLM+PDDL and LLM+ASP (B. Liu et al. 2023; Dagan et al. 2023; Guan et al. 2023; K. Yang et al. 2023).

Neural planners are trained models that learn planning behavior from data, for example, through reinforcement learning or imitation learning. Models such as DRRN and Decision

Transformer show that neural models can learn effective planning strategies in specific domains (He et al. 2016; Lili Chen et al. 2021). However, they may generalize poorly when training data are limited. Therefore, methods such as CALM and SwiftSage combine neural planners with LLM, using the neural planner for efficient action selection and LLM for more complex reasoning (Yao, Rao, et al. 2020; K. Lin et al. 2023).

Reflection and refinement are another planning-related task that allows the agent to evaluate whether a generated plan is correct, feasible, or needs improvement. In contrast to memory reflection, which mainly updates stored knowledge, planning reflection focuses on revising the current plan based on errors, feedback, or intermediate results. This enables the agent to refine weak or incomplete plans before continuing with the task. However, this process can also increase the computational cost, as additional reasoning steps are required (Xu Huang et al. 2024).

Memory-augmented planning uses previously stored information to support the generation and adaptation of plans. Instead of planning only from the current prompt, the agent can retrieve relevant past experiences, task history, or contextual knowledge from memory. This can improve the quality of the plan, especially in long-term tasks. However, the usefulness of this

approach depends on the relevance and quality of the retrieved memory (Xu Huang et al. 2024).

Action module.

The last module, the *action module*, works in conjunction with the previous modules and is responsible for carrying out the task plans generated by the planning module. While the planning module decides what should be done, the action module executes these decisions through concrete actions. In this way, the agent moves from reasoning and planning to actual task execution.

The consequences produced by these actions are referred to as *action impact*. These impacts can be described at different levels. First, actions may affect the external environment by changing its current state. Second, the outcome of an action may also affect the agent itself, for example, by updating its memory, revising its plans, or improving its understanding of the task. Finally, actions can create conditions for future actions, because completion of one step often enables the agent to continue with the next step (Park et al. 2023).

The action module therefore completes the agent cycle by turning the generated plan into an interaction with the environment and feeding the resulting outcome back into the system to inform subsequent decisions.

2.8 From single-agent to multi-agent LLM workflows

Architecturally, a single-agent system (SAS) centralizes retrieval, reasoning, and output generation within one workflow. This design is relatively simple and efficient, since it avoids coordination overhead and can reuse context within a single inference loop (X. Li et al. 2024). For this reason, recent studies (Mishra, Krishna, and al. 2024) and (Marcelloni et al. 2026) argue that SAS should be treated as a strong architectural baseline rather than as a trivial point of comparison. In particular, recent benchmark work shows that a strong single-agent implementation can match or even exceed the performance of homogeneous multi-agent workflows in several settings, while retaining an efficiency advantage due to lower coordination and inference overhead (Wu, Cheng, and al. 2024). This suggests that the value of MAS cannot be assumed a priori but must be demonstrated under controlled conditions.

At the same time, SAS can become limited when one model is expected to retrieve heterogeneous evidence, perform multi-step reasoning, maintain consistency across intermediate steps, and synthesize a final answer within a single loop. This challenge motivates the shift toward multi-agent systems (MAS), in which mul-

multiple agents share responsibility for different subtasks. Recent conceptual work distinguishes between narrower AI agents and more advanced agentic AI systems, where multiple specialized agents cooperate through dynamic task decomposition, structured interaction, and shared workflow goals. In this sense, MAS do not simply add more model instances, but introduce a different architectural principle in which reasoning is distributed across specialized roles.

The transition from SAS to MAS is, therefore, motivated by the need to handle tasks that exceed the practical limits of a single generalist workflow. The prior literature suggests that MAS can improve decomposition, perspective-taking, and intermediate validation, especially in settings that require diverse forms of reasoning or multiple knowledge sources. However, these gains are accompanied by a new design challenge: once reasoning is distributed across agents, communication and orchestration become central components of system performance. Consequently, the comparison between SAS and MAS is not only a question of task accuracy, but also of how much architectural complexity is justified by the resulting gains.

2.9 Multi-Agent Systems

In contrast to Single-Agent Systems (SAS), Multi-Agent Systems (MAS) extend the capabilities of LLM-based agents by introducing interaction, cooperation, and coordination between multiple agents. While an SAS relies mainly on the internal modules of one agent, such as profile, memory, planning, and action, a MAS focuses on how several agents can work together to solve more complex tasks. This allows the system to distribute responsibilities, combine different capabilities, and simulate forms of teamwork or social interaction. However, constructing an MAS is more challenging, as communication, coordination, and task distribution between agents need to be carefully designed.

MAS are especially useful when a task becomes too complex for a single agent or when different capabilities are required to achieve a reliable result. In this setting, the system can benefit from multiple agents who contribute different perspectives, skills, or responsibilities. Different MAS architectures can be designed on the basis of homogeneous or heterogeneous agents. In a homogeneous MAS, agents have similar roles or capabilities, while in a heterogeneous MAS, agents are designed with different profiles, tools, or responsibilities. This design difference allows MAS to address a wider range of tasks and pro-

motes complementarity and synergy between agents.

The following section extends the agent lifecycle previously introduced for SAS by focusing on the elements that are specific to MAS, such as *agent composition*, *shared perception*, *communication*, *coordination*, *task decomposition*, *mutual interaction*, and *evolution* (X. Li et al. 2024). Figure 5 provides an overview of these substeps in the MAS architecture.

Agent Composition and Role Distribution

The first step in building an MAS is to define the problem that the system aims to solve so that the number, type, and characteristics of the agents can be determined. In contrast to SAS, where the profile defines the behavior of one agent, in MAS the profile also determines how responsibilities are distributed across multiple agents. Each agent can be designed with specific functional capabilities, structural features, and behavioral strategies, depending on the task and the application scenario (L. Wang et al. 2024).

A common approach is the *predefined method*, where multiple agents are created in advance and form an agent pool. Based on the specific scenario, the most suitable agents are then selected. This method enables a faster and less resource-intensive process compared to regenerating similar



Figure 5: Multi-Agent System (MAS) architecture. The figure presents the main stages of a multi-agent workflow, from shared planning and information exchange to reasoning, action, and feedback.

agents every time a complex task is executed. Another approach is the *learning-based method*, where a few agents initiate the task and, when new situations arise, additional agents are generated to handle the new circumstances. This approach takes advantage of the capabilities of LLMs, as agents can be generated during task execution and tailored to evolving situational demands (Dong et al. 2024).

For example, in an RAG-based enterprise workflow, one agent may be responsible for retrieving relevant documents, another for filtering noisy context, another for generating the answer, and another for validating whether the answer is grounded in the retrieved evidence (Guohao Li et al.

2023).

However, this flexibility also introduces challenges. Dynamically generated agents may suffer from hallucinations, inconsistent profiles, or a mismatch between their assigned role and actual task requirements. Therefore, the design of agent roles is not only a profiling issue but also a coordination issue because the system needs to ensure that the agents complement each other instead of duplicating work.

Shared Environment and Distributed Perception

In MAS, perception is not limited to the information received by an individual agent. Instead, percep-

tion becomes distributed across several agents that may observe the same environment, receive different input, or interpret the same task from different perspectives. The shared environment represents the space in which agents operate and exchange information. Depending on the application, this environment can include a simulated scenario, a software system, external tools, databases, APIs, or a shared conversational workspace.

According to [X. Li et al. \(2024\)](#), agents can perceive information from three main sources. First, *environment messages* provide information about the surrounding context, such as location, spatial layout, time-related changes, or general atmosphere of a scenario. In a technical setting, this can also correspond to the state of a system, data source, or workflow. Second, *interaction messages* refer to the information exchanged between agents during communication, coordination, or collaboration. These messages influence how agents understand each other and decide their next actions. Third, *self-reflection messages* are internal messages through which an agent reviews its past actions, previous interactions, and environmental information. This allows the agent to update its understanding and improve its future behavior.

In general, perception in MAS describes how information flows between agents, the environment, and the in-

ternal state of each agent. This is important because the quality of the agents’ decisions depends not only on their individual perception but also on how effectively relevant information is shared across the whole system.

Communication and Coordination

As mentioned previously, one of the main challenges in MAS is the design of communication protocols and coordination mechanisms. These mechanisms need to ensure that agents exchange information efficiently, follow a coherent strategy, and maintain important system properties such as stability, scalability, and security.

Communication and coordination are the core mechanisms that allow agents to work together effectively. Although each agent can perceive, learn, and decide autonomously, its knowledge and capabilities are limited when operating in isolation. Communication helps close this gap by allowing agents to share information, exchange intermediate results, provide feedback, and synchronize their actions. As a result, the overall system can achieve better performance than a single isolated agent.

The *communication mechanism* in MAS can be described through three main elements. First, *communication protocols* define the rules that govern how agents exchange information,

including the mode, frequency, and sequence of communication. Second, the *communication topology* defines the connectivity structure between agents and determines which agents communicate directly and which rely on intermediaries. Third, the *communication content* describes what is actually exchanged, which can include natural language, structured data, code, task instructions, or intermediate results.

The *coordination mechanism* describes how agents align their behavior in order to achieve individual or collective goals. In cooperation, agents pursue shared goals by combining resources, information, and tasks. In competition, agents pursue optimal outcomes under resource constraints, often through game-theoretic or bidding approaches. In negotiation, agents resolve conflicts over goals, resources, or decisions through dialog and mutual concessions.

Task Decomposition and Orchestration

A central advantage of MAS is the ability to decompose complex tasks into smaller subtasks and assign them to specialized agents. While SAS planning describes how one agent breaks down and solves a task internally, task decomposition in MAS focuses on how responsibilities are distributed across multiple agents. Each agent can then

work on a specific part of the problem according to its role, capability, or access to tools.

The orchestration mechanism defines how these subtasks are assigned, executed, monitored, and combined to obtain a final result. In some architectures, orchestration can be centralized, where a coordinator agent manages the workflow and decides which agent should act next. In other architectures, orchestration can be more decentralized, where agents communicate directly and coordinate their actions through mutual interaction. The choice of orchestration strategy influences the efficiency, flexibility, and maintainability of the MAS.

This aspect is particularly important in complex technical environments, where one agent may be responsible for analyzing input data, another for identifying quality problems, another for suggesting preprocessing steps, and another for validating the final result. In this way, MAS can transform a complex task into a structured workflow composed of multiple specialized contributions.

Individual Action within the Multi-Agent Workflow

Although MAS focuses on interaction between agents, each agent still needs to perform individual actions based on its role, memory, perception, and planning capabilities. However, the mean-

ing of action changes in an MAS context because the output of one agent often becomes the input of another agent. Therefore, individual actions are not isolated, but are part of a broader collaborative workflow.

For example, one agent may generate an intermediate result, while another agent reviews, corrects, or extends it. This creates a feedback loop in which agents not only act independently, but also influence each other’s future decisions. As a result, the quality of the MAS depends not only on the capabilities of each individual agent but also on how well their actions are connected, evaluated, and integrated into the final system output.

Collective Feedback and Validation

Another important characteristic of MAS is the possibility to use *collective feedback and validation*. Instead of relying on one agent to generate and evaluate its own output, different agents can take complementary roles in the reasoning process. For example, one agent may propose a solution, another agent may critique it, and a third agent may validate whether the result satisfies the task requirements.

This mechanism can improve the robustness of the system, because errors, inconsistencies, or missing information can be detected through in-

teraction between agents. However, it also increases the need for clear coordination rules, as unstructured feedback can lead to repeated discussions, conflicting output, or unnecessary computational cost.

Evolution

Evolution describes how agents or the overall MAS can adapt over time based on feedback, previous interactions, and changes in the environment. In an MAS, evolution concerns not only the improvement of a single agent but also the improvement of the interaction patterns, role distribution, coordination strategy, and shared workflow of the system.

In a multi-agent environment, reflection can also be coordinated at the system level. For example, a dedicated coordinator agent may support or control the reflection process of individual agents to maintain coherence across the system. This can help align the agents’ memories, reduce inconsistencies, and improve coordination between their future actions. Therefore, evolution is important because it allows MAS to become more coherent, adaptive, and useful in repeated interactions ([X. Li et al. 2024](#)).

2.10 Toward Efficient and Filtered Multi-Agent RAG

While multi-agent systems (MAS) are often associated with improved performance on complex tasks, recent work suggests that part of these gains may be attributable to increased inference and communication budgets rather than to superior workflow design alone. In particular, recent benchmark research shows that a strong single agent system (SAS) can match the performance of many homogeneous multi-agent workflows while retaining a substantial efficiency advantage through KV cache reuse and reduced orchestration overhead (J. Xu et al. 2026). This finding suggests that MAS should not be assumed to be superior by default but should be evaluated against strong SAS baselines under controlled conditions.

A related line of work further highlights that architectural design choices, such as orchestration depth, execution semantics, and interface structure, can have a decisive impact on system behavior. Framework-level benchmarking demonstrates that these choices alone can induce substantial differences in latency, throughput, planning precision, and coordination success (Orogat, Rostam, and Mansour 2026). This reinforces the view that MAS efficiency is not merely a matter of model quality, but a

broader system design problem.

Recent studies have therefore begun to explicitly focus on reducing the communication overhead of MAS. SupervisorAgent, for example, introduces a modular runtime supervision framework with an LLM-free adaptive filter that intervenes at critical points in the workflow. Rather than changing the base agent architecture, the framework selectively corrects inefficient behavior and filters observations before they propagate further. The reported results show that this approach can reduce token consumption while maintaining task success in benchmark settings such as question answering, code generation, and mathematical reasoning (F. Lin et al. 2025). Similarly, the work on communication redundancy in consensus-based MAS identifies unnecessary interaction patterns as a central source of inefficiency and proposes structured communication mechanisms that reduce unnecessary exchanges while preventing the precision of reasoning (L. Yang, S. Li, and Deng 2026). Role-oriented communication further limits information sharing to role-relevant content instead of allowing unrestricted message passing (Xie et al. 2026).

Beyond communication itself, efficiency also depends on how agent responsibilities and participation are organized. Role-based pipelines assign complementary responsibilities to specialized agents (Lima et al. 2026),

while debate-based systems rely on iterative critique to improve reasoning and coordination (Du et al. 2024). However, these approaches generally define the participating agents and interaction structure in advance. More dynamic approaches instead adapt the composition of the workflow according to the requirements of the current task.

Agent-Level Filtering: Dynamic Team Selection

One approach to reducing unnecessary multi-agent overhead is therefore to

dynamically select which agents participate in a task. DyLAN (Z. Liu et al. 2023) addresses the limitation of static multi-agent systems, where the same number of agents and communication structure may be used regardless of the task. Since not every agent is equally relevant for every problem or communication round, DyLAN dynamically reorganizes the participating team according to task requirements. Inspired by human teamwork, the framework allows different agents to become more or less relevant depending on the problem being solved.

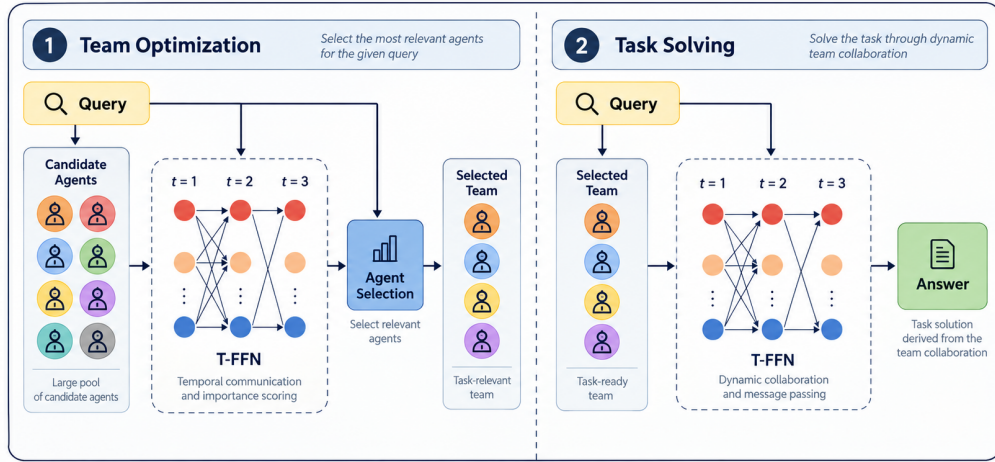


Figure 6: DyLAN: dynamic agent team selection using temporal feed-forward networks (T-FFNs). Adapted from DyLAN (Z. Liu et al. 2023). The figure illustrates how task-relevant agents are dynamically selected and coordinated to solve a given query efficiently.

The central idea of DyLAN is to formulate agent collaboration as a *Temporal Feed-Forward Network* (T-FFN), illustrated in Figure 6. In this representation, each communication round corresponds to one network layer. Agents are represented as nodes, while communication channels between agents are represented as edges. A set of candidate agents is defined as

$$\mathcal{A} = \{a_1, a_2, \dots, a_N\},$$

where each agent a_i can be an LLM-powered agent, possibly equipped with tools, or an independent tool. At a specific time step t , the nodes are defined as:

$$\mathcal{V}_t = \{v_{t,1}, \dots, v_{t,N}\},$$

where $v_{t,i}$ represents agent a_i at time step t . The temporal edge set between two consecutive time steps is defined as:

$$E_{t-1,t} = \{(v_{t-1,i}, v_{t,j})\} \subseteq \mathcal{V}_{t-1} \times \mathcal{V}_t.$$

These edges indicate which agents can communicate between two consecutive rounds.

DyLAN operates in two stages. The first stage is *Team Optimization*. In this stage, the framework performs a preliminary collaboration using the pool of candidate agents and then estimates the contribution of each agent through an unsupervised metric called

the agent importance score. This score is inspired by backward message passing and neuron importance methods, and is used to select a smaller task-oriented team from the original agent pool. In this way, DyLAN does not rely only on manually defined roles but evaluates which agents actually contribute to solving the task.

The second stage is *Task Solving*. During this stage, the selected agents collaborate through forward message passing on the T-FFN. For a node $v_{t,j}$, the messages received from adjacent nodes are aggregated into an updated message:

$$\hat{m}_{v_{t,j}} = f_{\text{mp}}(\mathcal{M}, v_{t,j}),$$

where $\mathcal{M}$ denotes the set of incoming messages and f_{mp} is the message aggregation function. In practice, this means that an agent receives the responses of other agents from the previous round and generates a new response based on the task query and the messages received. This response can include criticism, refinement, advice, or quality assessment, depending on the role of the agent.

A further important component of DyLAN is the reformation of agent teams. During task solving, an LLM-based ranker evaluates the responses of the participating agents and selects the top-ranked agents for the next time step. Therefore, agents that performed low or were less relevant are

deactivated in subsequent communication rounds. This creates a dynamic communication structure instead of a fixed one.

The relevance of DyLAN for the present thesis lies in its demonstration that multi-agent performance depends not only on the number of agents but also on selecting the most relevant participants and organizing their communication effectively. Empirical results show that DyLAN can outperform several static multi-agent baselines across reasoning, arithmetic, decision-making, and code-generation tasks, with agent selection improving accuracy by up to 25% % in selected subjects of the Massive Multitask Language Understanding (MMLU) benchmark (Z. Liu et al. 2023). These findings indicate that dynamically adapting agent participation can reduce unnecessary coordination while preserving, and in some cases improving, multi-agent reasoning performance.

Communication-Level Filtering: Topology and Message Pruning

A related line of work focuses not on selecting agents, but on optimizing the communication structure between them. G-Designer models a multi-agent system as a graph, where agents are represented as nodes and communication links as edges. Instead of relying on a fixed topology, the framework dynamically designs task-aware com-

munication structures, demonstrating that the organization of agent interactions can influence both performance and efficiency (G. Zhang et al. 2025).

Similarly, AgentPrune addresses the same efficiency problem from a different perspective. Rather than constructing a new communication topology, it removes redundant or low-contribution interactions from the spatio-temporal message-passing graph. By reducing unnecessary message exchanges, the approach aims to reduce token consumption while preserving task performance (T. Zhang 2025). Together, these approaches show that the efficiency of MAS depends not only on which agents participate but also on how information is exchanged between them.

Dynamic Agent and Edge Elimination

AgentDropout further extends this idea by dynamically eliminating both agents and communication links during multi-agent collaboration. The method is motivated by the observation that not every agent contributes equally in every communication round. AgentDropout represents the MAS as a communication graph and optimizes the graph structure by applying two mechanisms: *Node Dropout* and *Edge Dropout*. Node Dropout removes agents with a lower estimated contribution, while Edge Dropout re-

moves redundant or low-value communication links between agents. In this way, the framework reduces unnecessary participation and message passing, improving token efficiency while preserving or improving task performance (TODO Wang 2025).

It is important to distinguish this approach from fine-tuning the LLM itself. AgentDropout does not train the internal weights of the language model. Instead, it optimizes weights associated with the communication topology, which are used to decide which agents and edges should remain active. This distinction is relevant for the present thesis since the proposed filtered MAS does not train model weights or topology weights. Instead, it adopts the broader idea of selective participation and implements it through lightweight inference-time filtering of retrieved context and inter-agent communication.

Context-Level Filtering in RAG

While the previous approaches focus mainly on agents and their communication structure, RAG-based systems introduce an additional filtering problem: not all retrieved documents or chunks are useful for generation. Passing irrelevant retrieved context to the LLM can increase token usage, distract the model from relevant evidence, and contribute to unsupported or weakly grounded answers. There-

fore, filtering in RAG can be applied at the level of retrieved context before it enters the generation or multi-agent reasoning process.

MAIN-RAG addresses this issue by proposing a multi-agent filtering approach for retrieval augmented generation (Chang 2025). Instead of directly passing all retrieved documents to the generator, multiple agents evaluate and filter the retrieved evidence prior to answer generation. This is relevant for the present thesis because it connects the idea of multi-agent collaboration with the specific problem of noisy RAG context.

A complementary direction is context compression. SARA introduces a hybrid compression strategy that balances fine-grained evidence preservation with global abstraction using compression vectors (Jin 2026). This line of work shows that improving RAG is not only a matter of retrieving more context, but also of selecting, compressing, and preserving the parts of the context that are actually useful for the final answer.

2.11 Positioning of this Thesis

Existing studies show that efficiency can be improved by filtering agents, communication, or retrieved context, but these mechanisms are often examined separately and mainly on general-purpose benchmarks. Therefore, it re-

mains unclear how selective coordination performs in an enterprise analytical setting, where agents must combine semantic knowledge, retrieval, SQL generation, and evidence-based answer generation.

This thesis addresses this gap through a controlled comparison of four architectures ranging from a single-agent baseline to a dynamically filtered multi-agent system with RAG. Rather than learning communication topologies or modifying the parameters of the underlying language model, the proposed approach applies lightweight filtering during inference to control the retrieved semantic context and the participation of specialized agents. The objective is to investigate whether this selective allocation of context and reasoning can reduce computational overhead while preserving the quality and groundedness of the answer .

3. Research Methodology

This chapter describes the experimental methodology used to implement and evaluate the four LLM-based architectures considered in this study. First, it presents the research design and experimental setting, followed by the four architectures, the experimental procedure, and the evaluation metrics used for comparison.

3.1 Research Design

The experiment follows a controlled comparative design in which the agent architecture represents the independent variable. All four architectures process the same natural language questions using the same underlying data, semantic knowledge base, language model, SQL execution environment, and evaluation procedure. The underlying resources remain consistent across architectures, while the amount of semantic information made available during each run varies according to the architecture design. Each architecture is evaluated on the same set of 25 questions in three repetitions. This setup enables question by question comparison while reducing the influence of factors not related to the analytical architecture. The evaluation set covers different levels of analytical complexity. Some questions can be answered using information from a single table, while others require filtering and joining multiple tables, calculating metrics, or comparing different results. This variation allows architectures to be evaluated across tasks with different reasoning and data-processing requirements.

The four architectures are organized as incremental variations of a common analytical workflow. The **Single-Agent System (SAS)** serves as the baseline and receives the complete technical and semantic context

without retrieval. **SAS_RAG** preserves the single-agent workflow, but introduces question-specific retrieval. **MAS_RAG** uses the same retrieval resources while distributing the analytical process among five specialized agents. Finally, **Dynamic Filtered MAS_RAG** extends the multi-agent workflow by adapting agent participation and the amount of contextual information according to the requirements of the question. The main characteristics of the four architectures are summarized in Table 1.

The incremental design enables for three controlled comparisons. SAS and SAS_RAG are compared to assess the effect of replacing the complete semantic context with question-specific retrieval. SAS_RAG and MAS_RAG are then compared to examine the effect of task decomposition, role specialization, and multi-agent coordination while retaining the same retrieval mechanism. Finally, MAS_RAG and Dynamic Filtered MAS_RAG are compared to determine whether dynamic filtering can reduce the coordination overhead of the static multi-agent architecture without reducing answer quality.

The resulting differences are evaluated using three groups of metrics: claim-level response quality, reliability, and operational and computational performance. The quality of the answers is assessed through precision, recall, and the F1 score, while reli-

ability includes groundedness, hallucination risk, and abstention. Operational and computational performance is evaluated by evaluating SQL execution success, latency, and token consumption. Considering these dimensions together allows the experiment to examine whether improvements obtained through multi-agent coordination justify the additional computational cost.

3.2 Data and Experimental Setting

The data set is derived from an e-learning platform that operates in the healthcare sector. It consists of ten pseudonymised relational tables, including users, companies, courses, assignments, assignment course progress, certificates, exam results, contract-related information, and sales opportunities, with the learning-related data covering the period 2022 to 2026. The resulting tables were stored in a Unity Catalog schema within the Databricks environment.

Before the experiment, the data set was preprocessed to remove known data-quality issues so that the evaluation would focus on the analytical capabilities of the architectures rather than on data-cleaning tasks.

Two complementary forms of contextual information from the database structure were prepared: a techni-

Characteristic	SAS	SAS_RAG	MAS_RAG	Dynamic Filtered MAS_RAG
Agents	1	1	5	2–5
Workflow	Single-agent	Single-agent with retrieval	Fixed five-agent workflow	Adaptive multi-agent workflow
Semantic context	Complete semantic layer	Question-specific retrieval	Question-specific retrieval	Retrieved and filtered context
Retrieval	None	VS ($k = 5$, $\tau = 0.30$)	VS ($k = 5$, $\tau = 0.30$)	VS + evidence filtering
Quality auditor	✗	✗	✓	✓
Maximum SQL retries	0	0	1	2
Legend: ✓ = present, ✗ = not present; VS = vector similarity; k = maximum number of retrieved semantic units; τ = similarity threshold.				

Table 1: High-level comparison of the four evaluated architectures.

cal schema and a semantic layer. The technical schema describes which database objects are available and how they are structurally related, including tables, columns, data types, and join relationships. The semantic layer complements this structural information by describing the business meaning of the available data and how they should be interpreted for analytical purposes. It contains table and column descriptions, metric definitions, semantic join rules, and domain-specific business rules.

The semantic layer was assembled programmatically from existing metadata sources and standardized prior to evaluation.

Model inference for all four architectures was performed using the `databricks-meta-llama-3-1-8b-instruct` model through the Databricks model serving environment.

3.3 Agent Architectures

Single-Agent System (SAS)

The Single-Agent System (SAS) serves as the baseline and represents the simplest architecture evaluated in this study. It consists of a single agent acting in the role of a data analyst and is responsible for processing the complete analytical workflow.

For every question, the agent receives the complete technical schema together with the full semantic layer.

This ensures that all curated information is available to the agent, although part of the provided context may be irrelevant to the specific question being processed. Unlike the retrieval-based architectures introduced later, the contextual information supplied to SAS is therefore not selected according to the question. Figure 7 illustrates this difference by contrasting the fixed context delivery of SAS with the question-specific retrieval used in SAS_RAG.

The workflow consists of four sequential stages. First, during the planning stage, the agent interprets the question and develops an analytical plan based on the available technical and semantic information. In the second stage, the same agent translates this plan into an SQL query. The generated query is then executed programmatically against the experimental database.

There are three possible statuses of the result of the query:

1. The query produces a non-empty table.
2. The query produces an empty table.
3. The SQL query fails.

In the final stage, the agent receives the information produced throughout the previous stages, including the original question, the analytical plan, the generated SQL query,

and its execution result. Based on this information, the LLM generates the final answer in natural language. The resulting responses are subsequently assessed against the reference answers using the metrics defined in the evaluation section.

As the baseline architecture, SAS does not use retrieval, inter-agent communication, or conversational memory. Each question is independently processed, and the same complete technical and semantic context is provided for every evaluation case. The subsequent architectures extend this baseline by introducing retrieval, multi-agent specialization, and dynamic filtering.

Retrieval-Augmented Single-Agent Architecture (SAS_RAG)

The Single-Agent System with Retrieval-Augmented Generation (**SAS_RAG**) adds a retrieval stage to the SAS workflow, allowing the agent to receive the selected semantic information for the specific question before planning begins. While SAS receives the complete technical schema and the semantic layer for every question, SAS_RAG retains the complete technical schema but selects only the semantic information considered relevant to the current question. The main architectural difference therefore concerns how the semantic context is provided to the



Figure 7: Comparison between the Single-Agent System (SAS) and the retrieval-augmented single-agent architecture (SAS_RAG).

agent.

The semantic layer consists of 53 documents, which are embedded once before the evaluation using the **databricks-gte-large-en** embedding model. When a new question is received, it is embedded using the same model and compared with the stored document representations by cosine similarity. The retriever returns at most five documents and only documents with a similarity score of at least 0.3 are retained. The retrieval depth ($k = 5$) and the similarity threshold ($\tau = 0.30$) were heuristically selected and kept constant in all retrieval-based architectures.

As a result, SAS_RAG receives selected semantic information for the individual question, whereas SAS re-

ceives the complete semantic layer in every run.

The retrieved semantic documents are then combined with the complete technical schema and provided to the agent. From this point on, the analytical workflow remains identical to SAS, following the same four stages of analytical planning, SQL generation, programmatic SQL execution, and interpretation of the results. The model parameters, stage-specific prompt templates, SQL restrictions, and abstention conditions are also kept unchanged. This design isolates the delivery of semantic information as the main difference between SAS and SAS_RAG.

Multi-Agent Architecture with RAG (MAS_RAG)

The RAG Multi-Agent Architecture (MAS_RAG) extends SAS_RAG by distributing the analytical workflow across a fixed sequence of five specialized agents rather than assigning all analytical tasks to a single agent. The retrieval mechanism and the underlying technical and semantic resources remain unchanged from SAS_RAG. The principal architectural difference lies in how the reasoning process is organized between specialized roles.

The architecture follows a static sequential structure in which all five agents participate in a predefined order unless the workflow terminates early. The roles were selected to reflect the main analytical tasks required by the enterprise data and evaluation questions. Each agent is defined by a specific role and a role-specific system prompt describing its responsibilities and expected behavior.

1. **Domain Expert:** Interprets the question from the perspective of the enterprise learning and compliance domain, identifies relevant business terminology and domain-specific rules, and assesses whether the question can be answered using the available data.
2. **Data Engineer:** Maps the in-

terpreted business requirements to the available data structure by identifying the relevant tables, columns, and relationships required for the analysis.

3. **SQL Developer:** Translates the previously defined business and data requirements into an executable SQL query.
4. **Quantitative Analyst:** Interprets the SQL results and performs the numerical analysis required to answer the question, including additional calculations and comparisons where necessary.
5. **Quality Auditor:** Reviews the available evidence and assesses whether the proposed answer is factually and numerically supported by the executed query results.

Each role is implemented through a separate invocation of the same underlying language model. Agents differ only in their role-specific instructions and predefined decoding configurations.

Communication follows a cumulative structure in which each agent receives the original question, the technical schema, the retrieved semantic information, and the outputs produced by the preceding agents. The Data

Engineer therefore receives the interpretation of the Domain Expert, while the SQL Developer receives both the domain interpretation and the resulting data-model guidance. Following query execution, the generated SQL statement and its tabular result are additionally made available to the

Quantitative Analyst and Quality Auditor. All intermediate outputs are maintained in a shared collaboration record and propagated to the subsequent stages. Figure 8 illustrates this sequential information flow.

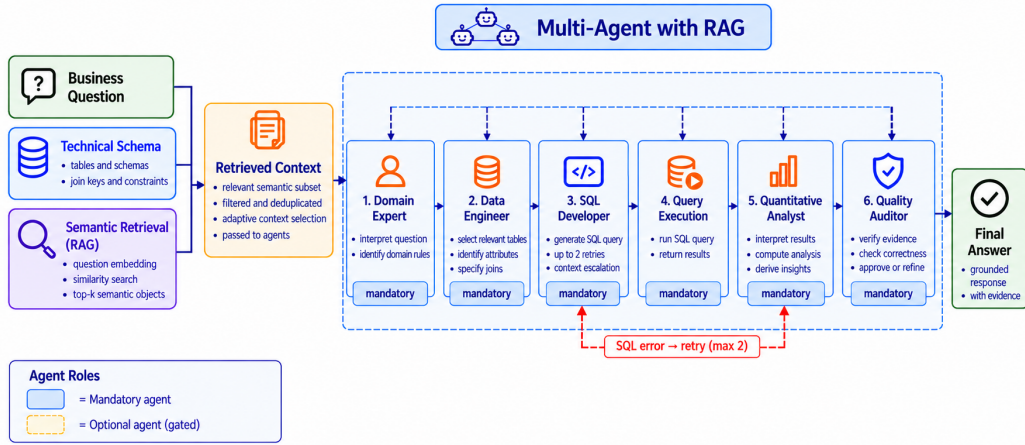


Figure 8: Multi-Agent System with Retrieval-Augmented Generation (MAS-RAG).

If SQL execution fails, the execution error is returned to the SQL Developer, which is allowed one additional attempt to revise the query. A maximum of two SQL-generation attempts is therefore permitted. If the second attempt also fails, the SQL stage is marked as unsuccessful, and downstream agents are instructed not to derive numerical or factual results from the failed execution. This safeguard is intended to prevent unsup-

ported interpretations that could otherwise increase the risk of hallucinated responses.

The Quality Auditor performs the final validation before an answer is returned. If the available evidence is insufficient or the proposed response contains unsupported factual or numerical claims, the architecture abstains rather than producing a substantive answer.

A complete MAS-RAG workflow re-

quires five LLM invocations, or six when the SQL Developer performs a correction attempt, compared to three in **SAS_RAG**. Since intermediate outputs are passed to subsequent agents, the amount of context also increases throughout the workflow, contributing to additional computational overhead.

MAS_RAG serves as the static multi-agent baseline for the dynamically filtered architecture introduced next.

Dynamic Filtered Multi-Agent Architecture with RAG

The static MAS_RAG architecture follows a fixed sequence of specialized agents regardless of the requirements of the individual question. In contrast, the **Dynamic Filtered Multi-Agent Architecture with RAG** introduces agent gating, selective information transfer, evidence filtering, and early stopping or bypass rules to avoid unnecessary processing.

The objective is to preserve the reasoning capabilities of MAS_RAG while reducing unnecessary invocations of the model, token usage, and latency.

Figure 9 provides an overview of the resulting dynamic workflow and its filtering stages.

Unlike the static MAS_RAG architecture, not all five agents are mandatory in the dynamic workflow. The SQL Developer and Quality Auditor

form the mandatory core, while the Domain Expert, Data Engineer, and Quantitative Analyst are included according to their activation scores.

In addition to these adaptive mechanisms, the architecture retains the same specialist roles and general analytical sequence as MAS_RAG. When an agent is activated, it performs the same role-specific task as in the static architecture, while the dynamic mechanism determines whether the agent is required and how much information is provided during execution.

The dynamic behavior of the architecture is controlled by a three-tier gating mechanism. Based on the characteristics of the question, the mechanism calculates an activation score for each agent and uses this score to determine whether the agent is skipped, operates in compressed mode, or operates in full mode.

The first step extracts a five dimensional binary feature vector from each question. The vector represents the characteristics of the question that are relevant to determining which specialist agents may be required.

$$\mathbf{x} = [1 \quad d \quad m \quad r \quad c]^\top.$$

The constant value 1 represents a general activation component, ensuring that each agent receives a base activation weight independently of the characteristics of the question. The

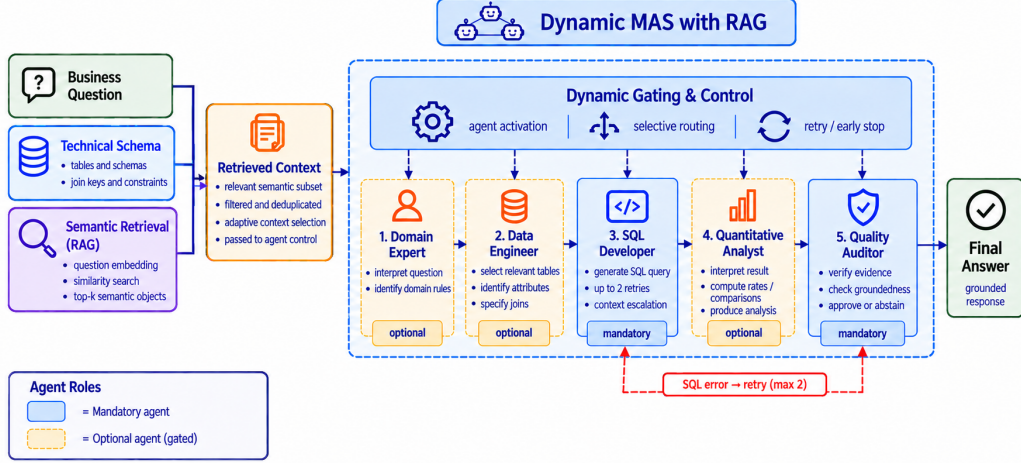


Figure 9: Dynamic Filtered Multi-Agent System with Retrieval-Augmented Generation.

remaining features capture properties that may increase the relevance of particular specialist agents. The feature d captures domain-specific terminology through a predefined keyword list. The feature m identifies questions that require multiple tables or join operations. The feature r identifies questions involving rates or percentages, and c represents comparative or ranking operations. Each feature takes a binary value and is detected by matching deterministic keyword with predefined word lists applied to the question text.

In the second step, the extracted feature vector is used to calculate an activation score for each agent. Each agent a_i is associated with a predefined weight vector $\mathbf{w}_i \in \mathbb{R}^5$, which

reflects how relevant each characteristic of the question is to the respective specialist.

The weights are manually predefined according to the intended role of each specialist and remain fixed throughout the evaluation. The activation score is calculated from the dot product between the agent specific weight vector and the characteristic vector of the question and is bounded between 0 and 1, as shown in Eq. (11).

$$s_i = \mathbf{w}_i^\top \mathbf{x}. \quad (11)$$

The resulting score is subsequently used by the gating mechanism to determine the level of participation of each optional agent.

The final step applies the gating rules to determine how each optional

agent participates in the workflow. A score below 0.3 causes the agent to be skipped, while agents that pass the selection threshold operate in compressed or full mode according to their score:

$$\begin{aligned} s_i < 0.3 &\rightarrow \text{skipped,} \\ 0.3 \leq s_i < 0.6 &\rightarrow \text{compressed,} \\ s_i \geq 0.6 &\rightarrow \text{full.} \end{aligned}$$

Scores between 0.3 and 0.6 result in compressed mode, in which the agent is instructed to return a shorter structured JSON output instead of unrestricted free text. Scores of 0.6 or higher activate the full mode, in which the agent follows the same role specific behavior as in the static MAS_RAG architecture. In this way, agents with moderate relevance can still contribute while consuming fewer input and output tokens.

In addition to adapting agent participation, the architecture also controls the amount of semantic evidence provided to the workflow.

The evidence filtering stage processes the semantic chunks retrieved from the vector store before they are provided to the agents. The near duplicate chunks are first removed using a Jaccard similarity threshold above 0.92, after which the chunks with a relevance score below 0.35 are excluded. The remaining evidence is then subject to an adaptive context budget

based on retrieval quality, ranging from 500 to 1,500 tokens. Higher retrieval similarity results in a smaller evidence budget, while lower similarity allows additional semantic units to be retained. This mechanism limits unnecessary context, while allowing broader evidence to be considered when retrieval is less certain.

An additional technique introduced in this architecture is communication pruning. In the MAS_RAG architecture, each agent receives the output of all previous predecessors. Instead, this architecture uses a direct routing map, which passes only the output of specific predecessors that are relevant to its task. Another filtering mechanism introduced by the dynamic architecture is communication pruning. In the static MAS_RAG architecture, each agent receives the outputs of all the preceding agents. In contrast, the dynamic architecture uses a predefined routing map that passes only the outputs of the preceding agents that are relevant to the task of the receiving agent.

The architecture additionally introduces an SQL retry mechanism with context escalation. If the first SQL execution fails, the error message is provided to the SQL Developer for a second attempt. If this attempt also fails, a third and final attempt is performed with escalated context, giving the SQL Developer access to the full outputs of the preceding agents

rather than only the pruned information. This mechanism allows the architecture to restore additional context when the filtered information is insufficient for successful SQL generation.

Finally, the architecture applies early stopping and bypass rules to avoid unnecessary processing when further analysis cannot provide additional value. If the SQL Developer exhausts all three execution attempts without producing a valid query result, the architecture abstains, and the remaining downstream processing is skipped because no reliable result is available for interpretation.

In addition, when the SQL execution returns a compact result consisting of at most one row and two columns, the Quantitative Analyst can be bypassed because no additional quantitative interpretation is required. The Quality Auditor remains active to validate the final response.

In general, the dynamic architecture preserves the analytical structure of MAS-RAG while adapting the amount of reasoning, semantic context, and inter-agent communication to the requirements of each question. The combination of agent gating, evidence filtering, communication pruning, retry escalation, and early stopping is intended to reduce unnecessary computational overhead without removing the specialized reasoning capabilities of the multi-agent workflow.

3.4 Experimental Procedure

The experiments are run through a Databricks notebook pipeline controlled by a central orchestrator. For each evaluation question, the orchestrator applies the selected architecture, prepares the required context, executes the corresponding LLM workflow, and runs any generated SQL against the approved Unity Catalog tables. The final response and the information required for subsequent evaluation are then stored automatically.

To ensure consistent and safe execution, all architectures are restricted to read-only SQL. Only **SELECT** statements are permitted, while commands such as **UPDATE**, **DELETE**, and **DROP** are rejected before execution.

3.5 Evaluation Metrics

The four architectures are evaluated using a set of complementary metrics commonly applied in previous studies of RAG-based systems and related LLM (L. Chen et al. 2025; Saleh 2025; Zhu, Tong Wang, Yixuan Li, et al. 2025). Since no single metric can capture all aspects of system performance, the evaluation is divided into three groups.

First, answer quality is assessed through metrics that capture correctness and completeness. A second group of metrics focuses on halluci-

nation risk and abstention behavior. Finally, the third group evaluates computational performance, with particular attention to the resources required by each architecture.

Answer quality

The first group of quality metrics comprises precision, recall, and the F1 score. Ru et al. [Ru et al. \(2024\)](#) proposes a fine grained approach in which the generated answers are decomposed into atomic factual statements, with each statement representing one independently verifiable claim. Afterwards, a claim entailment checker compares the claims extracted from the generated response with those extracted from the ground truth answer. Figure 10 illustrates this claim level comparison and distinguishes between correct, incorrect, and missing claims.

Precision measures the proportion of generated claims that are supported by the ground truth answer, while recall measures the proportion of ground truth claims covered by the generated response. The F1 score combines both measures through their harmonic mean and, therefore, provides a balance between correctness and completeness. The F1 score is calculated as shown in Eq. (12).

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (12)$$

Together, these three metrics are used to evaluate the claim-level task performance of each architecture.

Reliability Metrics

Groundedness measures the extent to which the claims generated by the architecture are supported by the available evidence rather than relying only on the model’s internal knowledge. It is calculated as the proportion of generated claims that can be supported by the retrieved semantic information or SQL results, as shown in Eq. (13). Here, C_{sup} denotes the set of supported claims, while C_{gen} denotes the set of all claims contained in the generated answer.

$$\text{Groundedness} = \frac{|C_{\text{sup}}|}{|C_{\text{gen}}|} \quad (13)$$

Hallucination risk measures the proportion of numerical claims in the generated answer that cannot be traced to the SQL result. For this purpose, numerical values are extracted from both the final answer and the SQL output using regular expressions. In addition, small deviations caused by rounding or formatting are accepted within a predefined tolerance, as unsupported non numerical content is not captured by this metric.

$$\text{HR} = \frac{|N_{\text{unsup}}|}{|N_{\text{sup}}| + |N_{\text{unsup}}|} \quad (14)$$

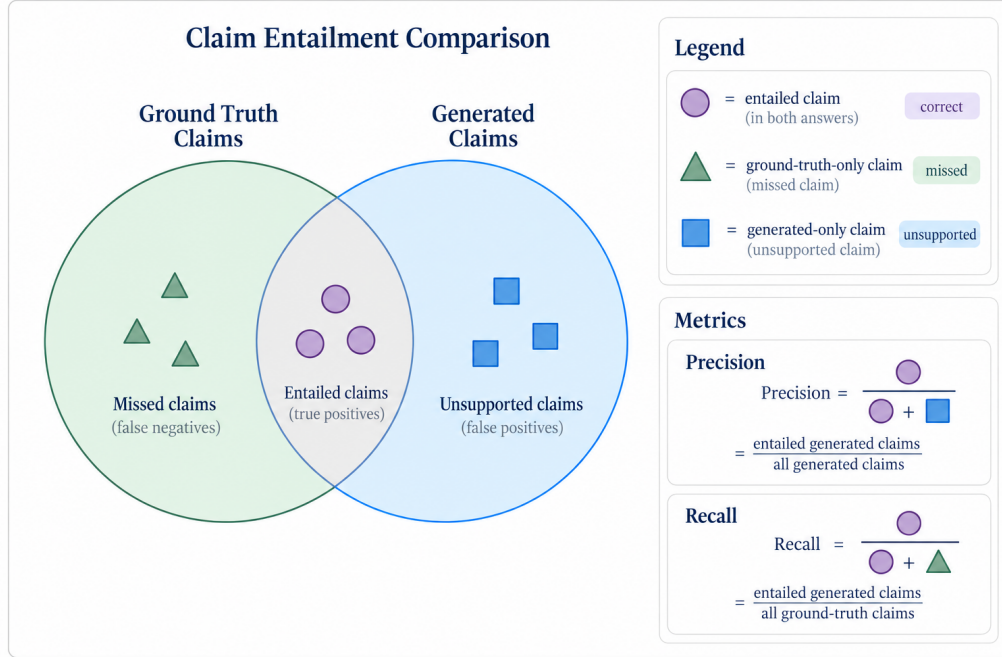


Figure 10: Overview of the evaluation metrics used in the experiments. The figure illustrates the claim-level comparison between the generated and ground-truth answers. Correct, missing, and unsupported claims form the basis for calculating precision and recall.

Here, N_{unsup} denotes the set of unsupported numerical claims, while N_{sup} denotes the set of numerical claims supported by the SQL result. Groundedness and hallucination risk therefore assess different aspects of reliability and are not expected to sum up to one.

Abstention measures whether an architecture correctly avoids answering when the available information is insufficient. For intentionally unanswerable questions, refusing to answer is considered a correct abstention. In

contrast, abstinence from an answerable question is considered a false abstention because the required information is available in the experimental data.

This distinction is important because a system could reduce hallucinations simply by refusing to ask too many questions. Abstention is therefore evaluated together with the other quality metrics.

Operational and Computational Metrics

The final group of metrics evaluates the operational reliability and computational efficiency of the four architectures. Include SQL execution success, latency, and token consumption.

The SQL execution success rate measures how often a generated SQL query executes without a runtime error and returns a valid result. It does not evaluate whether the returned data is semantically correct, but indicates whether the architecture is able to produce executable analytical queries. As shown in Eq. (15), the metric is calculated as the number of successfully executed queries ($N_{\text{successful}}$) divided by the total number of SQL queries executed during the evaluation ($N_{\text{SQL runs}}$).

$$\text{SQL Success Rate} = \frac{N_{\text{successful}}}{N_{\text{SQL runs}}} \quad (15)$$

Latency measures the total time required, in seconds, by an architecture to process a question and produce the final response. The measurement starts when architecture-specific processing begins and includes all intermediate stages, such as retrieval, where applicable, agent processing, SQL generation and execution, and final answer generation.

Token consumption measures the number of tokens used by the LLM

during each run. Input and output tokens are recorded separately and summed to obtain the total token usage, based on the values returned by the model-serving endpoint.

4. Results

The following section is the last part of the thesis, where the experimental findings of the four architectures and their performance in 25 domain-specific analytical questions regarding a company’s learning management system (LMS) will be explained in detail.

4.1 Overall Performance Across Architectures

Table 2 provides an overview of the overall performance of all four architectures in terms of response quality, SQL execution, and computational efficiency.

As explained in the evaluation metrics section, the comparison between architectures should consider multiple metrics together, since relying on a single quality measure could lead to a distorted interpretation of the results.

Starting with the comparison between SAS and SAS_RAG, the aim is to isolate the effect of introducing retrieval-augmented generation (RAG) into the single-agent system. Overall, the two architectures achieve similar answer quality, although Figure 11(b) shows a different pattern across difficulty levels. SAS_RAG

Table 2: Overall performance of the four evaluated architectures.

Metric	SAS	SAS_RAG	MAS_RAG	Dynamic MAS_RAG
Claim Precision	0.18	0.20	0.22	0.27
Claim Recall	0.11	0.12	0.19	0.30
Claim F1	0.11	0.10	0.17	0.26
Groundedness	0.62	0.60	0.40	0.79
Hallucination Risk	0.56	0.48	0.37	0.20
Abstention Rate	1/25 (5%)	0/25 (0%)	4/25 (17%)	18/25 (71%)
SQL Success Rate	9/25 (35%)	7/25 (29%)	11/25 (43%)	8/25 (33%)
Avg. Latency (s)	6.0	5.9	14.3	9.3
Avg. Tokens per Question	17,591	12,062	40,880	21,926

Values are averaged across three repetitions. Bold indicates the preferable result where applicable.

performs substantially better on easy questions, reaching more than twice the F1 score of SAS. However, this advantage disappears as question difficulty increases, with SAS_RAG showing a stronger decline on medium and hard questions while SAS remains comparatively more stable. The strong performance on easy questions therefore compensates for weaker results at higher difficulty levels, leading to a similar overall F1 score. This suggests that retrieved context is particularly useful when the question aligns well with the available semantic information, while its benefit becomes more limited for complex analytical tasks.

A second important difference appears in the handling of unanswerable questions. Across the 15 runs corresponding to the five unanswerable questions, SAS_RAG did not abstain and attempted to provide an an-

swer in every case. This resulted in a hallucination risk of 1.0 for this subset. SAS showed more conservative behavior, abstaining in approximately 20% of these cases. Groundedness, in contrast, remained nearly unchanged between the two architectures, indicating that there were no substantial overall differences in the evidential support of their generated claims.

The clearest benefit of SAS_RAG is computational efficiency. Although the overall quality of the answer remains close to the SAS baseline, SAS_RAG reduces the average token consumption by approximately 31%. The retrieval step therefore reduces the amount of context processed by the LLM without producing a substantial change in overall F1. However, this efficiency gain is accompanied by weaker abstention behavior on unanswerable questions, showing that retrieval alone does not ensure that

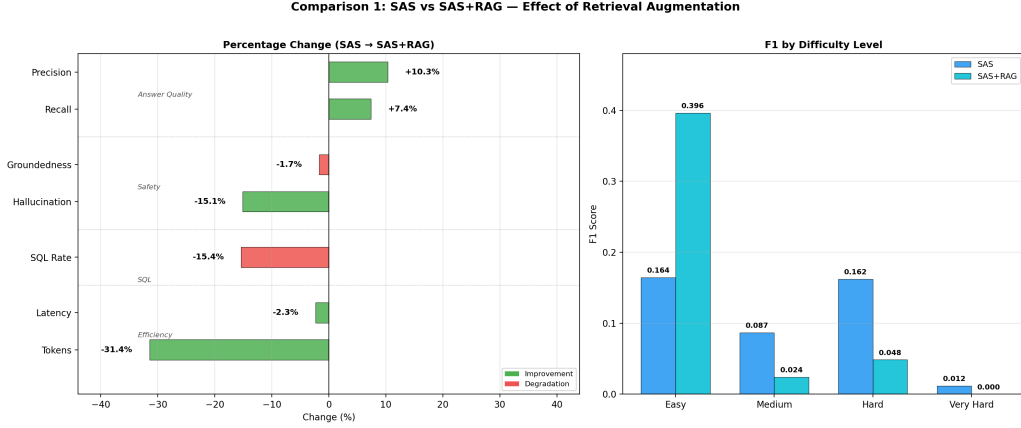


Figure 11: Comparison of SAS and SAS+RAG. (a) Relative change across the main evaluation metrics. (b) F1 scores by question difficulty level.

the available evidence is sufficient to support an answer.

The next comparison examines SAS+RAG and MAS+RAG to determine whether multi-agent collaboration improves answer quality enough to justify the additional computational cost. MAS+RAG shows substantial improvements in most answer-quality metrics. The F1 score increases by approximately 70%, driven by improvements in both precision and recall, while hallucination risk decreases by roughly 23%. The multi-agent architecture also handles unanswerable questions more cautiously, increasing the correct abstention rate from 0% to 33%.

However, these improvements come at a substantial computational cost. As shown in Figure 12, latency more than doubles and token con-

sumption roughly triples because the multi-agent workflow requires several LLM calls for each question. The groundedness also decreases, indicating that although MAS+RAG generates more correct claims in general, a higher proportion of these claims are not directly supported by the available evidence.

A further trade-off appears in the abstention behavior. Although MAS+RAG correctly refuses more unanswerable questions, it also abstains on 8 of the 60 answerable runs (13%), compared to no false abstentions for SAS+RAG. In general, task decomposition and role specialization improve claim-level accuracy and reduce hallucination risk compared to the single-agent RAG architecture, but these gains are accompanied by lower groundedness and considerably

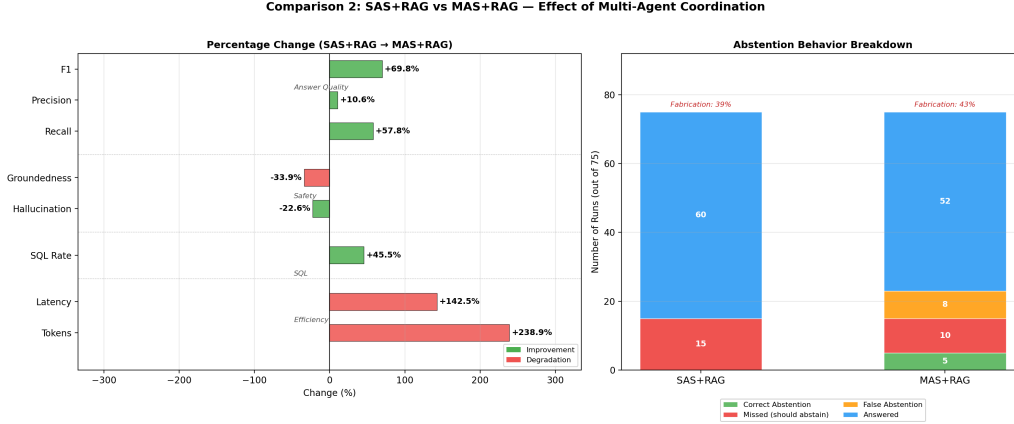


Figure 12: Comparison of SAS_RAG and MAS_RAG. (a) Relative change across the main evaluation metrics. (b) Abstention behavior across answerable and unanswerable runs.

higher computational overhead.

The final comparison directly addresses the central hypothesis of this thesis by comparing MAS_RAG with Dynamic MAS_RAG. The purpose is to determine whether the filtering mechanisms of the Dynamic architecture can reduce the coordination overhead of the static multi-agent system without compromising answer quality.

Notably, the Dynamic architecture improves both answer quality and computational efficiency, a combination not observed in the previous comparisons. The F1 score increases by approximately 54%, from 0.170 to 0.263. The largest improvement is observed in groundedness, which nearly doubles, indicating that almost four out of five factual claims generated by the Dynamic system are supported

by the available semantic evidence or SQL results. This suggests that filtering irrelevant or less useful information is associated with more evidence supported responses. The risk of hallucination also decreases by approximately 46%, reaching the lowest value among the four architectures. At the same time, token consumption decreases by 46% and latency by 35%, reflecting the reduced amount of context and agent processing required by the dynamic workflow. An exception is SQL success, which decreases from 43% to 33%, indicating that the efficiency gains do not extend uniformly to SQL generation.

Relative to the SAS baseline, the Dynamic architecture achieves approximately 2.4 times the F1 score while using only about 1.25 times as many

tokens. This indicates a considerably more favorable quality cost relationship than the static MAS_RAG architecture, as illustrated in Figure 13.

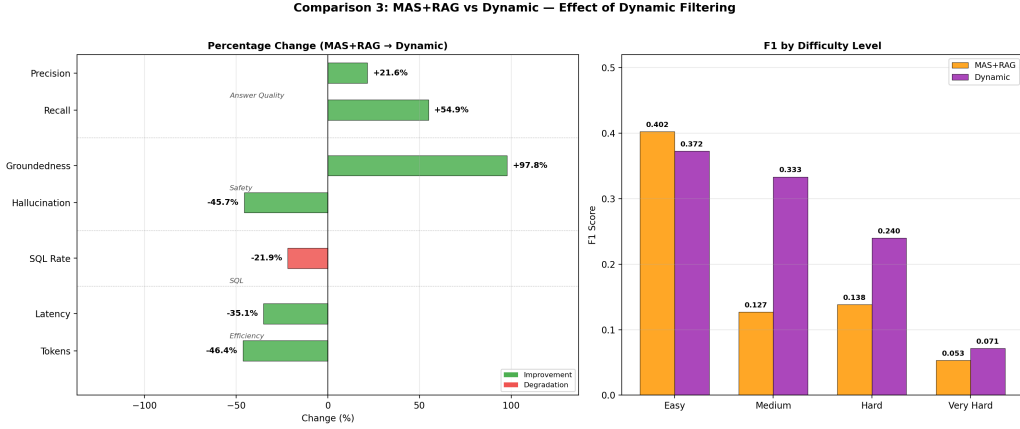


Figure 13: Comparison of MAS_RAG and Dynamic MAS_RAG. (a) Relative change across the main evaluation metrics. (b) F1 scores across question difficulty levels.

However, these improvements are accompanied by an important limitation. The Dynamic system behaves considerably more conservatively when deciding whether sufficient evidence is available to answer a question. It correctly abstains on 80% of unanswerable questions, compared to 33% for MAS_RAG, and reduces post-failure fabrication from 43% to 4%. At the same time, 41 of the 60 answerable runs (68%) result in abstention, compared to 13% for MAS_RAG. This high false-abstention rate is particularly relevant for harder questions and shows that the filtering and stopping mechanisms can become overly restrictive. Despite this limitation,

the Dynamic architecture combines higher claim-level quality and groundedness with lower hallucination risk, token consumption, and latency, resulting in the most favorable quality efficiency trade-off among the evaluated architectures.

5. Conclusion

This thesis examined whether the benefits of multi-agent coordination justify its cost in analytical question answering over enterprise data. By evaluating the quality, computational effort, and coverage of the response together, it shows why the value of an architecture cannot be captured by a single performance measure. In the

setting studied, selective coordination reduced the overhead of a multi-agent workflow while improving measured answer quality. However, the same system frequently rejected questions that the available data could answer. The central hypothesis is therefore supported with an important qualification: filtering can make collaboration more efficient, but its value depends on preserving the ability to return supported answers when sufficient evidence is available.

5.1 Limitations and future work

The study points out a number of limitations that need to be recognized. Firstly, the evaluation is confined to one area, namely a corporate learning management system, and to one base model, Meta-Llama-3.1-8B-Instruct. Because of this, the results cannot easily be applied to other areas or to models that have different reasoning abilities. Secondly, the benchmark includes only 25 questions, each of which is assessed three times. Although this arrangement offers some initial indication of differences in architecture, the small number of individual questions means that it is not possible to reach firm statistical conclusions.

Third, even though the researcher manually created the expected answers and the individual claims that made them up, the generated outputs

were assessed against this standard entirely by means of automated metrics at the claim level. Nothing beyond what is captured by the pre-defined claims was examined by any independent human assessors regarding the quality, relevance, or groundedness of the answers. Fourth, various parameters of the Dynamic architecture such as the activation thresholds, the filtering rules, and the routing configuration were set based on heuristic judgments rather than being optimized through experimental means. Although the degree of agent involvement and the flow of information vary depending on each question, the decision thresholds remain fixed. These decisions could have been a factor in the high rate of false abstentions on questions that were answerable.

Future research should therefore assess various architectures using larger and more diverse question sets, additional domains, and different language models, while also incorporating human evaluation alongside automated metrics. In the case of the Dynamic architecture, its filtering and routing mechanisms could be further improved by means of adaptive or data-driven calibration. Instead of setting the activation and filtering thresholds manually, they could be adjusted according to the complexity of the question, the confidence in the retrieval, the intermediate results, or the system’s historical performance

when larger evaluation datasets are available.

Further experiments might also be carried out to look at different arrangements of agents, various communication structures, and other retrieval settings in order to gain a better understanding of their impact on the compromise between quality and efficiency. Moreover, assessing a number of filtering configurations along the quality cost frontier could also give more practical advice as to when multi-agent coordination is advantageous and when a simpler architecture is enough.

Bibliography

- Arrais, Fernando et al. (2026). “Improving process systems engineering with specialized multi-agent large language models”. In: *Chemical Engineering Journal Advances*, p. 101141. DOI: [10.1016/j.cej.2026.101141](https://doi.org/10.1016/j.cej.2026.101141).
- Arslan, Muhammad, Hussam Ghanem, et al. (2024). “A Survey on RAG with LLMs”. In: *Procedia Computer Science* 246, pp. 3781–3790. DOI: [10.1016/j.procs.2024.11.000](https://doi.org/10.1016/j.procs.2024.11.000).
- Arslan, Muhammad, Saba Munawar, and Christophe Cruz (2024). “Sustainable Digitalization of Business with Multi-Agent RAG and LLM”. In: *Knowledge-Based and Intelligent Information & Engineering Systems: Proceedings of the 28th International Conference KES-2024*. Procedia Computer Science. Elsevier, pp. 4722–4731. DOI: [10.1016/J.PROCS.2024.09.337](https://doi.org/10.1016/J.PROCS.2024.09.337). URL: <https://doi.org/10.1016/j.procs.2024.09.337>.
- Ba, Jimmy Lei, Jamie Ryan Kiros, and Geoffrey E. Hinton (2016). “Layer Normalization”. In: *arXiv preprint arXiv:1607.06450*. URL: <https://arxiv.org/abs/1607.06450>.
- Bai, Yushi et al. (2025). “LongBench v2: Towards Deeper Understanding and Reasoning on Realistic Long-context Multitasks”. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, pp. 3639–3664. URL: <https://aclanthology.org/2025.acl-long.183>.
- Bender, Emily M. et al. (2021). “On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?” In: *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*. ACM, pp. 610–623.
- Besta, Maciej et al. (2024). “Graph of Thoughts: Solving Elaborate Problems with Large Language Models”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 38. 16, pp. 17682–17690. DOI: [10.1609/aaai.v38i16.29720](https://doi.org/10.1609/aaai.v38i16.29720).

- Cavnar, William B. and John M. Trenkle (1994). “N-Gram-Based Text Categorization”. In: *Proceedings of SDAIR-94, 3rd Annual Symposium on Document Analysis and Information Retrieval*. Las Vegas, NV, pp. 161–175. URL: <https://www.let.rug.nl/vannoord/TextCat/textcat.pdf>.
- Chang, TODO (2025). *MainRAG (placeholder entry)*. Placeholder to fix missing-citation warning. Please replace with the correct BibTeX (authors/venue/DOI/URL).
- Chen, L. et al. (2025). “Application of retrieval-augmented generation for interactive industrial knowledge management via a large language model”. In: *Computer Standards & Interfaces* 94, p. 103995. DOI: [10.1016/j.csi.2025.103995](https://doi.org/10.1016/j.csi.2025.103995).
- Chen, Lili et al. (2021). “Decision Transformer: Reinforcement Learning via Sequence Modeling”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: <https://arxiv.org/abs/2106.01345>.
- Chen, Xiaojin et al. (2025). “How is language intelligence evolving? A multi-dimensional survey of large language models”. In: *Artificial Intelligence Review*. DOI: [10.1007/s10462-025-10997-x](https://doi.org/10.1007/s10462-025-10997-x).
- Dagan, Guy et al. (2023). “Dynamic Planning with a Large Language Model”. In: *arXiv preprint arXiv:2305.07394*. URL: <https://arxiv.org/abs/2305.07394>.
- Devlin, Jacob et al. (June 2019). “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics, pp. 4171–4186. DOI: [10.18653/v1/N19-1423](https://doi.org/10.18653/v1/N19-1423). URL: <https://aclanthology.org/N19-1423>.
- Dietrich, Jens et al. (2003). “Rule-based agents for the Semantic Web”. In: *Electronic Commerce Research and Applications* 2.4, pp. 323–335. DOI: [10.1016/S1567-4223\(03\)00032-1](https://doi.org/10.1016/S1567-4223(03)00032-1).
- Dong, Yihong et al. (2024). “Self-Collaboration Code Generation via Chat-GPT”. In: *ACM Transactions on Software Engineering and Methodology* 33.5, pp. 1–29. ISSN: 1049-331X. DOI: [10.1145/3640330](https://doi.org/10.1145/3640330). URL: <https://arxiv.org/abs/2304.07590>.
- Du, Yilun et al. (2024). “Improving Factuality and Reasoning in Language Models through Multiagent Debate”. In: *Proceedings of the 41st International Conference on Machine Learning (ICML)*. URL: <https://proceedings.mlr.press/v235/du24e.html>.

- Gao, Luyu et al. (2023). “Precise Zero-Shot Dense Retrieval without Relevance Labels”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, pp. 1762–1777. URL: <https://arxiv.org/abs/2212.10496>.
- Gao, Yunfan, Yun Xiong, Xinyu Gao, et al. (2024). “Retrieval-Augmented Generation for Large Language Models: A Survey”. In: *arXiv preprint arXiv:2312.10997*.
- Gao, Yunfan, Yun Xiong, Meng Wang, et al. (2024). “Modular RAG: Transforming RAG Systems into LEGO-like Reconfigurable Frameworks”. In: *arXiv preprint arXiv:2407.21059*. URL: <https://arxiv.org/abs/2407.21059>.
- Guan, Jian et al. (2023). “Leveraging Large Language Models for Classical Planning”. In: *arXiv preprint arXiv:2305.16237*. URL: <https://arxiv.org/abs/2305.16237>.
- Al-Hammuri, Khalid et al. (2023). “Vision transformer architecture and applications in digital health: a tutorial and survey”. In: *Visual Computing for Industry, Biomedicine, and Art* 6.1, p. 14. DOI: [10.1186/s42492-023-00141-x](https://doi.org/10.1186/s42492-023-00141-x).
- Haslum, Patrik et al. (2019). *An Introduction to the Planning Domain Definition Language*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool Publishers. DOI: [10.2200/S00900ED2V01Y201902AIM042](https://doi.org/10.2200/S00900ED2V01Y201902AIM042).
- He, Ji et al. (2016). “Deep Reinforcement Learning with a Natural Language Action Space”. In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*. URL: <https://arxiv.org/abs/1511.04636>.
- Hu, Chenxu et al. (2023). “ChatDB: Augmenting LLMs with databases as their symbolic memory”. In: *arXiv preprint arXiv:2306.03901*. URL: <https://arxiv.org/abs/2306.03901>.
- Huang, Xiangji and Qinmin Hu (2009). “A Bayesian learning approach to promoting diversity in ranking for biomedical information retrieval”. In: *Proceedings of the 32nd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 307–314. DOI: [10.1145/1571941.1571995](https://doi.org/10.1145/1571941.1571995).
- Huang, Xu et al. (2024). “Understanding the planning of LLM agents: A survey”. In: *arXiv preprint arXiv:2402.02716*. URL: <https://arxiv.org/abs/2402.02716>.

- Huang, Yizheng and Jimmy Xiangji Huang (2023). “Diversified Prior Knowledge Enhanced General Language Model for Biomedical Information Retrieval”. In: *ECAI 2023 - 26th European Conference on Artificial Intelligence*. Vol. 372. Frontiers in Artificial Intelligence and Applications. IOS Press, pp. 1100–1108. DOI: [10.3233/FAIA230383](https://doi.org/10.3233/FAIA230383).
- (2026). “A Survey on Retrieval-Augmented Text Generation for Large Language Models”. In: *ACM Computing Surveys* 58.4, pp. 1–34. DOI: [10.1145/3805774](https://doi.org/10.1145/3805774). URL: <https://arxiv.org/abs/2404.10981>.
- Huang, Zuhao et al. (2023). “Memory Sandbox: Transparent and interactive memory management for conversational agents”. In: *Adjunct Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST ’23 Adjunct)*. New York, NY, USA: Association for Computing Machinery, pp. 1–3. DOI: [10.1145/3586182.3615801](https://doi.org/10.1145/3586182.3615801). URL: <https://dl.acm.org/doi/10.1145/3586182.3615801>.
- Ji, Ziwei et al. (2023). “Survey of Hallucination in Natural Language Generation”. In: *ACM Computing Surveys* 55.12, pp. 1–38.
- Jin, TODO (2026). *SARA (placeholder entry)*. Placeholder to fix missing-citation warning. Please replace with the correct BibTeX (authors/venue/DOI/URL).
- Kaplan, Jared et al. (2020). “Scaling Laws for Neural Language Models”. In: *arXiv preprint arXiv:2001.08361*. URL: <https://arxiv.org/abs/2001.08361>.
- Khattab, Omar and Matei Zaharia (2020). “ColBERT: Efficient and effective passage search via contextualized late interaction over BERT”. In: *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 39–48. DOI: [10.1145/3397271.3401075](https://doi.org/10.1145/3397271.3401075).
- Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. DOI: [10.48550/arXiv.2005.11401](https://doi.org/10.48550/arXiv.2005.11401). arXiv: [2005.11401](https://arxiv.org/abs/2005.11401). URL: <https://arxiv.org/abs/2005.11401>.
- Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33, pp. 9459–9474.

- Li, Guohao et al. (2023). “CAMEL: Communicative Agents for ”Mind” Exploration of Large Language Model Society”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: <https://arxiv.org/abs/2303.17760>.
- Li, Xinyi et al. (2024). “A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges”. In: *Vicinagearth* 1.1, p. 9. DOI: [10.1007/s44336-024-00009-2](https://doi.org/10.1007/s44336-024-00009-2).
- Lima, Fernando Arrais R. D. et al. (2026). “Improving process systems engineering with specialized multi-agent large language models”. In: *Chemical Engineering Journal Advances* 26, p. 101141. DOI: [10.1016/j.cej.2026.101141](https://doi.org/10.1016/j.cej.2026.101141).
- Lin, Fulin et al. (2025). “Stop Wasting Your Tokens: Towards Efficient Runtime Multi-Agent Systems”. In: *arXiv preprint arXiv:2510.26585*. URL: <https://arxiv.org/abs/2510.26585>.
- Lin, Kevin et al. (2023). “SwiftSage: A Generative Agent for Interactive Text-Based Environments”. In: *arXiv preprint*.
- Liu, Bo et al. (2023). “LLM+P: Empowering Large Language Models with Optimal Planning Proficiency”. In: *arXiv preprint arXiv:2304.11477*. URL: <https://arxiv.org/abs/2304.11477>.
- Liu, Pei, Xin Liu, Jun Ma, et al. (2025). “HM-RAG: Hierarchical Multi-Agent Multimodal Retrieval Augmented Generation”. In: *Proceedings of the 2025 ACM International Conference on Multimedia (MM ’25)*. ACM. URL: <https://arxiv.org/abs/2504.12330>.
- Liu, Zijun et al. (2023). “Dynamic LLM-Agent Network: An LLM-agent Collaboration Framework with Agent Team Optimization”. In: *arXiv preprint arXiv:2310.02170*. URL: <https://arxiv.org/abs/2310.02170>.
- Ma, Xinbei et al. (2023). “Query Rewriting for Retrieval-Augmented Large Language Models”. In: *arXiv preprint arXiv:2311.04854*. URL: <https://arxiv.org/abs/2311.04854>.
- Marcelloni, Francesco et al., eds. (2026). *Computational Intelligence: 17th International Joint Conference, IJCCI 2025, Marbella, Spain, October 22–24, 2025, Proceedings, Part I*. Vol. 2827. Communications in Computer and Information Science. Marbella, Spain: Springer Nature Switzerland. ISBN: 978-3-032-15631-0.
- McDermott, Drew et al. (1998). *PDDL—The Planning Domain Definition Language*. Tech. rep. CVC TR-98-003/DCS TR-1165. Yale Center for Computational Vision and Control.

- Mishra, Shubham, S. Krishna, and et al. (2024). “AI Agents vs. Agentic AI: A Conceptual Taxonomy, Applications, and Challenges”. In: *arXiv preprint arXiv:2409.11145*. URL: <https://arxiv.org/abs/2409.11145>.
- Modarressi, Ali et al. (2024). “Ret-LLM: Towards a general read-write memory for large language models”. In: *ICLR 2024 Workshop: How Far Are We From AGI?* Vienna, Austria. URL: <https://arxiv.org/abs/2405.14371>.
- Navigli, Roberto, Simone Conia, and Björn Ross (2023). “Biases in Large Language Models: Origins, Inventory and Discussion”. In: *ACM Journal of Data and Information Quality* 15.2, pp. 1–21. DOI: [10.1145/3597307](https://doi.org/10.1145/3597307).
- Orogat, Abdelghny, Ana Rostam, and Essam Mansour (2026). “Understanding Multi-Agent LLM Frameworks: A Unified Benchmark and Experimental Analysis”. In: *arXiv preprint arXiv:2602.03128*. URL: <https://arxiv.org/abs/2602.03128>.
- Park, Joon Sung et al. (2023). “Generative Agents: Interactive Simulacra of Human Behavior”. In: *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*. New York, NY, USA: Association for Computing Machinery, pp. 1–22. DOI: [10.1145/3586183.3606763](https://doi.org/10.1145/3586183.3606763). URL: <https://arxiv.org/abs/2304.03442>.
- Peng, Wang et al. (2023). “Large Language Model Based Long-Tail Query Rewriting in Taobao Search”. In: *arXiv preprint arXiv:2311.03758*. URL: <https://arxiv.org/abs/2311.03758>.
- Peters, Matthew E. et al. (2018). “Deep Contextualized Word Representations”. In: *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. Vol. 1, pp. 2227–2237. DOI: [10.18653/v1/N18-1202](https://doi.org/10.18653/v1/N18-1202).
- Prasad, Ishaan et al. (2024). “ADaPT: As-Needed Decomposition and Planning with Language Models”. In: *Findings of the Association for Computational Linguistics: NAACL 2024*. Association for Computational Linguistics. URL: <https://aclanthology.org/2024.findings-naacl.264/>.
- Qian, Chen et al. (2024). “ChatDev: Communicative Agents for Software Development”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, pp. 15174–15186. URL: <https://arxiv.org/abs/2307.07924>.
- Radford, Alec et al. (2018). “Improving Language Understanding by Generative Pre-Training”. In: *OpenAI Blog*. URL: <https://openai.com/research/language-unsupervised>.

- Ru, Dongyu et al. (2024). “RAGChecker: A Fine-grained Framework for Diagnosing Retrieval-Augmented Generation”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 37. URL: <https://arxiv.org/abs/2408.08067>.
- Saleh, TODO (2025). *Interoperable RAG (placeholder entry)*. Placeholder to fix missing-citation warning. Please replace with the correct BibTeX (authors/venue/DOI/URL).
- Shinn, Noah et al. (2023). “Reflexion: Language Agents with Verbal Reinforcement Learning”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 36, pp. 8634–8652. URL: <https://arxiv.org/abs/2303.11366>.
- Singh, Aditi et al. (2025). “Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG”. In: *arXiv preprint arXiv:2501.09136*. URL: <https://arxiv.org/abs/2501.09136>.
- Tamkin, Alex et al. (2022). “Task Ambiguity in Humans and Language Models”. In: *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 10118–10137. URL: <https://arxiv.org/abs/2212.10711>.
- Touvron, Hugo et al. (2023). “LLaMA: Open and efficient foundation language models”. In: *arXiv preprint arXiv:2302.13971*. URL: <https://arxiv.org/abs/2302.13971>.
- Vaswani, Ashish et al. (2017). “Attention is All You Need”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. URL: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.
- Vyas, Shivani et al. (2024). “Application of retrieval-augmented generation for interactive industrial knowledge management via a large language model”. In: *ResearchGate preprint*. DOI: [10.13140/RG.2.2.32561.35681](https://doi.org/10.13140/RG.2.2.32561.35681). URL: <https://www.researchgate.net/publication/394197368>.
- Wang, Lei et al. (2024). “A survey on large language model based autonomous agents”. In: *Frontiers of Computer Science* 18.6, p. 186345. DOI: [10.1007/s11704-024-40259-1](https://doi.org/10.1007/s11704-024-40259-1). URL: <https://arxiv.org/abs/2308.11432>.
- Wang, TODO (2025). *AgentDropout (placeholder entry)*. Placeholder to fix missing-citation warning. Please replace with the correct BibTeX (authors/venue/DOI/URL).
- Wang, Yidan, Jiayin Wang, and Zhiwei Chu (2025). “Multi-agent large language models as evolutionary optimizers for scheduling optimization”.

- In: *Computers & Industrial Engineering* 206, p. 111197. DOI: [10.1016/j.cie.2025.111197](https://doi.org/10.1016/j.cie.2025.111197).
- Wu, Tianshu, Yu Cheng, and et al. (2024). “Rethinking the Value of Multi-Agent Workflow: A Strong Single Agent Baseline”. In: *arXiv preprint arXiv:2402.16771*. URL: <https://arxiv.org/abs/2402.16771>.
- Xi, Jiannan et al. (2025). “The Rise and Potential of Large Language Model Based Agents: A Survey”. In: *Science China Information Sciences* 68.12, p. 121101. DOI: [10.1007/s11432-024-4222-0](https://doi.org/10.1007/s11432-024-4222-0).
- Xie, Zaipeng et al. (2026). “ROCO: Role-oriented communication for efficient multi-agent reinforcement learning”. In: *Expert Systems with Applications* 297, p. 129421. DOI: [10.1016/j.eswa.2025.129421](https://doi.org/10.1016/j.eswa.2025.129421).
- Xu, Jiawei et al. (2026). “Rethinking the Value of Multi-Agent Workflow: A Strong Single Agent Baseline”. In: *arXiv preprint arXiv:2601.12307*. URL: <https://arxiv.org/abs/2601.12307>.
- Yang, Ke et al. (2023). “Coupling Large Language Models with Logic Programming for Robust and General Reasoning from Text”. In: *Findings of the Association for Computational Linguistics: ACL 2023*. URL: <https://aclanthology.org/2023.findings-acl..>
- Yang, Liancheng, Sining Li, and Aobo Deng (2026). “Dynamic Consensus Communication Mechanism for Large Language Model-Based Multi-Agent Systems”. In: *Journal of Signal Processing Systems* 98.1, p. 10. DOI: [10.1007/s11265-025-01978-3](https://doi.org/10.1007/s11265-025-01978-3).
- Yao, Shunyu, Rohan Rao, et al. (2020). “Keep CALM and Explore: Language Models for Action Generation in Text-based Games”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, pp. 8736–8754. DOI: [10.18653/v1/2020.emnlp-main.704](https://doi.org/10.18653/v1/2020.emnlp-main.704).
- Yao, Shunyu, Dian Yu, et al. (2023). “Tree of Thoughts: Deliberate Problem Solving with Large Language Models”. In: *Advances in Neural Information Processing Systems*. Vol. 36.
- Zhang, Cong et al. (2024). “Planning with Multi-Constraints via Collaborative Language Agents”. In: *arXiv preprint arXiv:2405.16510*. URL: <https://arxiv.org/abs/2405.16510>.
- Zhang, Guibin et al. (2025). “G-Designer: Architecting Multi-agent Communication Topologies via Graph Neural Networks”. In: *Proceedings of the 42nd International Conference on Machine Learning (ICML)*.

- Zhang, Shengyu et al. (2025). “Instruction Tuning for Large Language Models: A Survey”. In: *ACM Computing Surveys* 57.11, pp. 1–38. DOI: [10.1145/3777411](https://arxiv.org/abs/2308.10792). URL: <https://arxiv.org/abs/2308.10792>.
- Zhang, TODO (2025). *AgentPrune (placeholder entry)*. Placeholder to fix missing-citation warning. Please replace with the correct BibTeX (authors/venue/DOI/URL).
- Zhang, Yue et al. (2023). “Siren’s Song in the AI Ocean: A Survey on Hallucination in Large Language Models”. In: *arXiv preprint arXiv:2309.01219*.
- Zhao, Wayne Xin et al. (2026). *Large Language Models*. Artificial Intelligence: Foundations, Theory, and Algorithms. Springer Nature Singapore. DOI: [10.1007/978-981-96-6259-3](https://doi.org/10.1007/978-981-96-6259-3).
- Zhao, Zirui, Wee Sun Lee, and David Hsu (2023). “Large Language Models as Commonsense Knowledge for Large-Scale Task Planning”. In: *Advances in Neural Information Processing Systems*. Vol. 36.
- Zhou, Xuanhe et al. (2023). “LLM as DBA: LLM-based autonomous agent for database administration”. In: *arXiv preprint arXiv:2308.05481*. URL: <https://arxiv.org/abs/2308.05481>.
- Zhu, Zekun, Tong Wang, Yixuan Li, et al. (2025). “Retrieval Augmented Generation Evaluation in the Era of Large Language Models: A Comprehensive Survey”. In: *arXiv preprint arXiv:2504.14891*. URL: <https://arxiv.org/abs/2504.14891>.